

Smart-Claims Agent

Sistema Agéntico de Procesamiento Multimodal y Ejecución Autónoma para la Gestión de Siniestros

Trabajo Fin de Máster — Entrega 2

Máster en Machine Learning e Inteligencia Artificial OBS Business School · Edición 2510 · Curso 2025–2026

Empresa de referencia: Seguros Pepín, S.A. (República Dominicana)

Equipo de trabajo:

- [Nombre y apellidos del integrante 1]
- [Nombre y apellidos del integrante 2]
- [Nombre y apellidos del integrante 3]
- [Nombre y apellidos del integrante 4]
- [Nombre y apellidos del integrante 5]
- [Nombre y apellidos del integrante 6]

Tutor: [Nombre del tutor]

Fecha de entrega: 26 de junio de 2026

Declaración de autoría

Los integrantes del equipo declaran que el presente trabajo es original y de su autoría, que se han citado adecuadamente todas las fuentes utilizadas conforme a la normativa APA 7.ª edición (Art. 7 de la normativa OBS), y que no incurre en plagio ni en uso indebido de materiales de terceros.

Firmas:

Integrante	Firma	Fecha
[Integrante 1]		
[Integrante 2]		
[Integrante 3]		
[Integrante 4]		
[Integrante 5]		
[Integrante 6]		

Índice de contenidos

1. **Arquitectura del sistema** — patrón Supervisor (Hub-and-Spoke), los seis agentes, flujo del expediente, gestión de estado, datos y conocimiento, capacidades de IA reales frente a integraciones simuladas, HITL y despliegue.

2. **Herramientas y capacidades** — herramientas `@tool` (mocks de sistemas externos) y capacidades de IA reales (extracción multimodal con Claude Vision, RAG de pólizas con ChromaDB, motor antifraude de cuatro detectores).
3. **Manual de usuario** — puesta en marcha (Docker, app Streamlit, CLI y API REST), operación, interpretación de resultados y resolución de problemas.
4. **Evaluación y resultados** — dataset sintético, protocolo, precisión, matriz de confusión, validación del motor antifraude y de la extracción multimodal.

\newpage

Capítulo 1 — Arquitectura del sistema

Arquitectura

1. Visión general del sistema

Smart-Claims Agent es un sistema agéntico orientado a automatizar el ciclo completo de gestión de una reclamación de siniestro de **Seguros Pepín, S.A.**, aseguradora real con sede en la República Dominicana, desarrollado como Trabajo Fin de Máster del Máster en Machine Learning e Inteligencia Artificial de OBS Business School. El objetivo funcional del sistema es recibir un expediente de siniestro, razonar de forma estructurada sobre él y resolverlo de manera autónoma —pago automático, rechazo, solicitud de información complementaria o derivación a revisión humana— dejando constancia auditable de cada decisión tomada por la cadena de agentes.

El alcance de esta entrega es deliberadamente el de un **MVP (Producto Mínimo Viable) demostrable**, no el de un sistema en producción. Esta decisión condiciona toda la arquitectura: prima la claridad, la reproducibilidad y la capacidad de demostración sobre la escalabilidad o la tolerancia a fallos de grado industrial. El sistema está concebido para ejecutarse de extremo a extremo y exhibir su razonamiento ante un tribunal académico.

Por su naturaleza, el sistema trata datos personales de asegurados (nombre, identificador de cliente, correo electrónico, documentación del siniestro). El marco normativo de referencia no es el RGPD europeo, sino la **Ley 172-13 de Protección de Datos de Carácter Personal de la República Dominicana**, que rige las garantías de tratamiento, conservación y minimización aplicables a Seguros Pepín. Esta condición se refleja en el diseño de la capa de datos (registro auditable, minimización de los datos persistidos) y en las salvaguardas de cumplimiento del Agente G.

Una **restricción clave y definitiva** atraviesa todo el diseño: el proyecto **no dispone de acceso a las APIs de los sistemas reales de Seguros Pepín** (gestor documental, sistema de pólizas, motor de pagos, listas oficiales de sanciones, canales de notificación). En consecuencia, **todas las integraciones con sistemas externos de la aseguradora están simuladas (*mock* / datos sintéticos) de forma permanente**, no como un estado transitorio a la espera de conexión. Esta restricción no es un defecto del prototipo, sino una condición de contorno asumida desde el inicio, y la arquitectura se ha diseñado para que la sustitución futura de los *mocks* por integraciones reales sea localizada y de bajo impacto.

Conviene precisar un matiz importante que recorre todo el capítulo: **el LLM utilizado (Claude) no es un sistema de Seguros Pepín**, sino una capacidad propia del proyecto contratada directamente a su proveedor. Por ello, mientras las integraciones corporativas permanecen simuladas, las **capacidades de inteligencia artificial son reales**: la extracción multimodal con visión por computador, la recuperación aumentada (RAG) sobre el corpus de pólizas y el razonamiento en lenguaje natural se ejecutan de verdad contra modelos y motores reales. La frontera entre "lo

simulado" y "lo real" se traza, por tanto, no entre IA y reglas, sino entre *integraciones con terceros sin acceso* (mock) y *capacidades de IA bajo control del proyecto* (reales). La sección 7 formaliza esta distinción.

En resumen, el sistema persigue tres propiedades rectoras que se repetirán a lo largo del capítulo: **reproducibilidad** (mismo expediente, misma traza), **trazabilidad / auditabilidad** (cada decisión queda registrada y justificada) y **resiliencia de la demostración** (el flujo funciona aunque falten componentes opcionales como la base de datos, la clave del LLM o el servicio vectorial).

2. El patrón Supervisor (Hub-and-Spoke) y su justificación

El sistema implementa un **patrón Supervisor**, también conocido como **Hub-and-Spoke**, sobre **LangGraph** (LangChain AI, 2024), materializado como un **grafo de estado** dirigido. El **Agente A actúa como supervisor central (hub)** y los agentes especializados (B, C, D, E, G) como radios (*spokes*). La propiedad clave del patrón es que **el supervisor es el único componente que decide el enrutamiento**: cada agente especializado realiza su tarea, escribe su contribución en el estado compartido y devuelve el control al supervisor, que vuelve a evaluar el estado y decide cuál es el siguiente nodo. Los agentes especialistas **no se llaman entre sí** en ningún caso.

Este patrón es una concreción de los sistemas multiagente basados en LLM descritos en la literatura reciente sobre agentes autónomos (Wang et al., 2024), donde la descomposición de una tarea compleja en subtareas asignadas a componentes especializados mejora la fiabilidad frente a un agente monolítico. LangGraph documenta explícitamente esta topología como uno de los patrones canónicos para sistemas multiagente (LangChain AI, 2024).

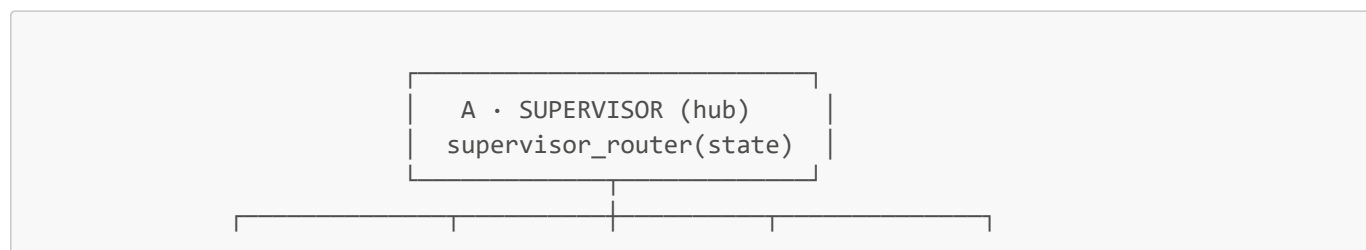
La alternativa natural habría sido un **bucle ReAct totalmente libre** (Yao et al., 2022), en el que un único agente alterna razonamiento y acción eligiendo dinámicamente qué herramienta invocar en cada paso. ReAct es un paradigma potente y flexible, pero presenta tres inconvenientes para los objetivos de este TFM:

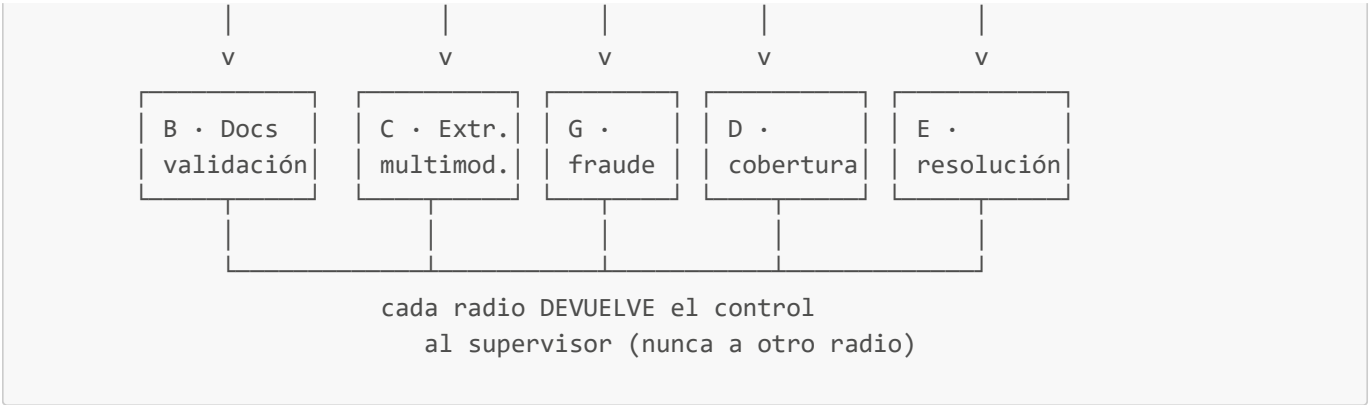
- **Reproducibilidad.** Un bucle libre puede tomar caminos distintos ante el mismo expediente, lo que dificulta una defensa ante tribunal en la que se espera un comportamiento estable y demostrable.
- **Control del coste de tokens.** Un agente que decide libremente cuántos pasos dar y qué herramientas usar tiene un consumo de tokens difícil de acotar; un grafo con nodos predefinidos hace ese coste predecible.
- **Trazabilidad y auditoría deterministas.** En un dominio asegurador, cada decisión debe poder explicarse y auditarse. Un grafo de estado con transiciones explícitas garantiza que la secuencia de decisiones sea determinista y reconstruible.

Por estas razones se ha optado por **fijar la topología del flujo en un grafo con supervisor central** en lugar de delegar la planificación en el propio modelo. El razonamiento del estilo ReAct no se descarta del todo: se conserva en el nivel de cada agente como **Chain of Thought** registrado, pero la *orquestración* —qué agente actúa después de cuál— es responsabilidad determinista del supervisor, no de una decisión libre del LLM. Se combina así la transparencia del razonamiento agéntico (Yao et al., 2022) con el control de un flujo de trabajo gobernado.

El núcleo del supervisor es una función Python pura y determinista, **supervisor_router**, que se invoca como arista condicional (*conditional edge*) del grafo. Decide el siguiente nodo a partir exclusivamente del estado acumulado, evaluando en orden si cada resultado parcial está disponible. Toda la lógica del flujo está concentrada en este único punto, lo que la hace especialmente legible, testeable (basta un test del router) y modificable.

El siguiente diagrama resume la topología Hub-and-Spoke:





3. Los seis agentes

El sistema se compone de **seis agentes** organizados en torno al supervisor. Cada agente reside en un fichero separado, siguiendo una convención de nomenclatura dual: el nombre del fichero refleja la responsabilidad funcional, y la letra del agente (A–G) aparece en *docstrings* y *logs* para mantener la trazabilidad con esta memoria.

Agente	Fichero	Rol	¿Real o simulado?
A	<code>orchestrator.py</code>	Supervisor + triaje. Punto de entrada del grafo; ejecuta el triaje inicial (razona el CoT) y aloja el <code>supervisor_router</code> que enruta a los demás agentes.	Lógica real; CoT con LLM real opcional.
B	<code>document_validator.py</code>	Validación documental. Verifica que el expediente contiene los tipos de documento requeridos según el tipo de siniestro; si faltan, solicita información y corta el flujo.	Lógica de validación real sobre el repositorio documental simulado.
C	<code>multimodal_extractor.py</code>	Extracción multimodal. Extrae tipo, importe, fecha, emisor, resumen y confianza de los documentos subidos (factura, foto, acta, PDF).	IA real con Claude Vision; <i>fallback</i> simulado si no hay archivos o clave de API.
G	<code>fraud_compliance.py</code>	Fraude y cumplimiento. Motor de cuatro detectores deterministas; emite veredicto graduado y marca el caso para HITL o bloqueo.	Motor real; lista de sanciones sintética que simula la SDN.
D	<code>coverage_checker.py</code>	Verificación de cobertura. Decide si el siniestro está cubierto recuperando la cláusula relevante de la póliza y citándola (límite, franquicia, neto pagable).	RAG real (ChromaDB); pólizas sintéticas; <i>fallback</i> determinista.
E	<code>claim_resolver.py</code>	Resolución autónoma. Determina la resolución final (PAGO, RECHAZO o REVISIÓN HUMANA) y la ejecuta.	Lógica real; ejecución de pago/rechazo simulada (<i>mock</i>).

A continuación se describe cada agente con el detalle necesario para fijar qué es real y qué es simulado.

A · Orquestador (`orchestrator.py`). Es el punto de entrada del grafo. Su nodo `triage_node` enriquece el estado con el razonamiento de bienvenida (sin decidir aún el enrutamiento) y deja el expediente listo para el primer

especialista. La función `supervisor_router`, descrita en la sección anterior, constituye el hub determinista del sistema.

B · Validación documental (`document_validator.py`). Comprueba que el expediente aporta los tipos de documento exigidos por el tipo de siniestro (por ejemplo, factura y acta para determinados casos). Si detecta documentación incompleta, redacta una solicitud de información al cliente y el supervisor corta el flujo: no es un rechazo, sino una pausa.

C · Extracción multimodal (`multimodal_extractor.py`, apoyado en `vision.py`). Es una de las **capacidades de IA reales** del proyecto. Sobre los documentos efectivamente subidos (factura, foto de daños, acta, PDF, informe de taller), invoca a **Claude Vision** (`claude-sonnet-4-6`) y extrae un objeto estructurado con tipo de documento, importe, fecha, emisor, resumen y confianza de la extracción. Como Claude no es un sistema de Seguros Pepín sino el LLM del propio proyecto, esta extracción es real. Si no se aportan archivos o no hay clave de API disponible, el agente recurre a una **extracción simulada** equivalente, de modo que el flujo nunca se interrumpe.

G · Fraude y cumplimiento (`fraud_compliance.py`, apoyado en `fraud_tools.py`). Implementa un **motor real de cuatro detectores deterministas**, diseñados para ser auditables señal a señal:

1. **Cribado de sanciones (OFAC / ONU)**. Compara el nombre del asegurado contra una lista mediante *fuzzy matching* (`difflib.SequenceMatcher`, umbral de similitud 0,82, con normalización de acentos). La lista es **sintética, de 15 entidades, y simula la lista SDN de la OFAC** y la lista de la ONU; en producción se descargaría periódicamente del servicio oficial.
2. **Anomalía de importe**. Calcula el *Z-score* del importe reclamado frente a una *baseline* por tipo de siniestro (umbral 2,0), marcando también el rebasamiento de un máximo legítimo.
3. **Duplicados recientes**. Detecta reclamaciones del mismo cliente y tipo dentro de una ventana de 90 días.
4. **Coherencia documental**. Verifica la consistencia temporal entre los documentos (fechas de siniestro, reclamación y factura), detectando incoherencias como una fecha de siniestro futura o una reclamación previa al siniestro.

Las señales se combinan en un **score compuesto** que produce un **veredicto graduado**: **BLOCKED** (coincidencia OFAC confirmada → rechazo automático), **HIGH_RISK** (score $\geq 0,55$ → HITL obligatorio), **MEDIUM_RISK** (score $\geq 0,25$ → HITL recomendado) y **CLEAR** (continúa el flujo). El expediente se marca para HITL o bloqueo cuando el veredicto es **HIGH_RISK** o **BLOCKED**.

D · Verificación de cobertura (`coverage_checker.py`, apoyado en `rag/policy_store.py`). Es la segunda **capacidad de IA real**: implementa **RAG (Retrieval-Augmented Generation)** (Lewis et al., 2020) con **ChromaDB embebido** (en proceso, sin servidor externo) sobre un corpus de pólizas (`data/policies/*.md`). El agente recupera la cláusula relevante mediante **búsqueda vectorial filtrada por el metadato `claim_type`** y decide la cobertura citando la sección recuperada (límite de cobertura, franquicia y neto pagable). Las pólizas son **sintéticas** (placeholder del prototipo; en producción se sustituirían por las condiciones reales de Seguros Pepín). Si el RAG no está disponible, el agente cae a una verificación determinista (`check_policy`), de modo que la demostración nunca se rompe.

E · Resolución (`claim_resolver.py`). Toma la decisión final del expediente —**PAGO, RECHAZO o REVISIÓN HUMANA**— y la ejecuta a través de *mocks* (autorización de pago o notificación de rechazo). La ejecución del pago es simulada porque depende del motor de tesorería de la aseguradora, al que no hay acceso.

LLM opcional (`reasoning.py`). Transversal a los agentes, el helper `reason()` enriquece el Chain of Thought de cada agente invocando a Claude (`claude-sonnet-4-6`) cuando la variable de entorno `ANTHROPIC_API_KEY` está disponible. En su ausencia, devuelve un *fallback* determinista que produce un razonamiento equivalente sin llamada externa. Esto garantiza una propiedad central: **la demostración funciona siempre**, con o sin conectividad al LLM, mitigando el riesgo de una caída de red o de cuota durante una defensa en vivo.

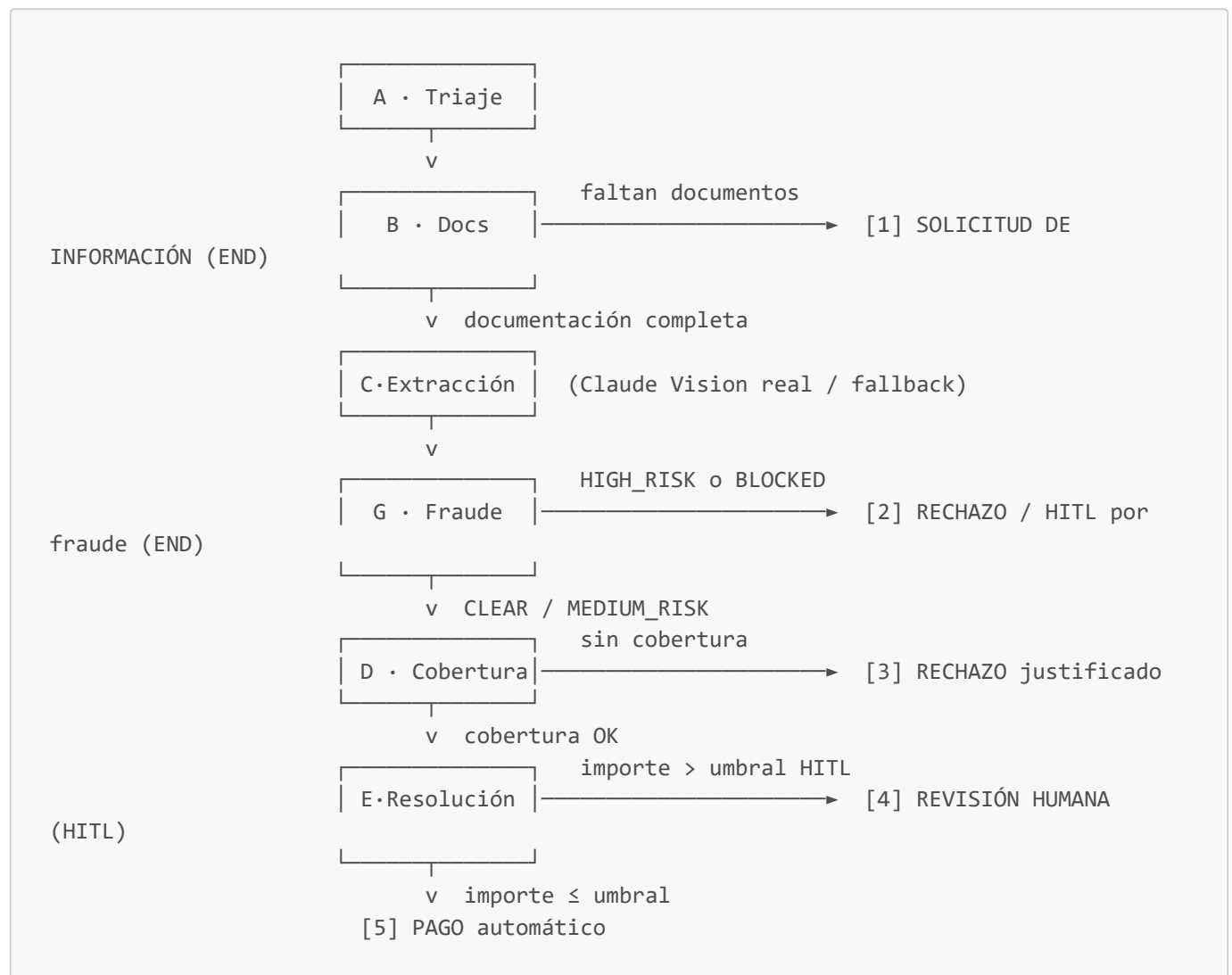
Agentes fuera del alcance del MVP. El catálogo conceptual del proyecto contemplaba además dos agentes que quedan **fuera del alcance de esta entrega** y se reservan para una fase posterior: el **Agente F** (predicción de judicialización del siniestro, basado en aprendizaje automático clásico sobre el histórico de expedientes) y el **Agente H** (asistente legal para expedientes judicializados). Esta entrega implementa el núcleo *end-to-end* del ciclo de gestión (A, B, C, D, E y G); la numeración no correlativa —se pasa de E a G— responde precisamente a esa reserva de las letras F y H para funcionalidades de una fase productiva posterior.

4. Flujo de una reclamación

El flujo nominal de extremo a extremo encadena los agentes en el orden:

A (triaje) → B (validación docs) → C (extracción) → G (fraude) → D (cobertura) → E (resolución)

Las transiciones las decide siempre el supervisor a partir del estado acumulado. Sobre esta espina dorsal se injertan **ramas condicionales** que pueden desviar el expediente antes de la resolución automática. El siguiente diagrama muestra el flujo completo con sus salidas:



Las **cinco salidas** del flujo son:

1. **Solicitud de información al cliente.** Si B determina que faltan documentos, el flujo termina con estado **validating** y decisión **INFO_REQUERIDA**. No es un rechazo, sino una pausa a la espera de información.

2. **Rechazo / HITL por fraude.** Si G emite veredicto `HIGH_RISK` o `BLOCKED`, el supervisor termina el flujo antes de la resolución. El expediente queda marcado como `rejected` con causa `RECHAZO_FRAUDE` (o derivado a revisión humana). Es el *gate* de cumplimiento.
3. **Rechazo justificado por no cobertura.** Si D concluye que el siniestro no está cubierto por la póliza, E redacta el rechazo citando la cláusula recuperada y lo notifica al cliente.
4. **Revisión humana por importe.** Si el siniestro tiene cobertura pero el importe supera el umbral HITL (`HITL_AMOUNT_THRESHOLD`, 5.000 € por defecto y configurable), E deriva el expediente a revisión humana antes de autorizar el pago.
5. **Pago automático.** Si el siniestro tiene cobertura y el importe es igual o inferior al umbral, E autoriza el pago de forma autónoma. Es el camino de máxima automatización.

4.1. Por qué el cribado de fraude (G) se ejecuta tras la recepción documental

Una decisión de diseño relevante es la **posición del Agente G en el flujo**. Aunque conceptualmente el cribado de fraude actúa como un *gate* de cumplimiento previo a la resolución y al pago, el router lo ejecuta **después** de la validación documental (B) y la extracción multimodal (C), y no como primer filtro de entrada. La razón es funcional: dos de los cuatro detectores de G —en particular el de **coherencia documental** y, en parte, el de **anomalía de importe**— necesitan los **datos efectivamente extraídos** de los documentos (fechas de siniestro, factura y reclamación; importe consolidado) para poder operar. Situar G antes de la extracción lo privaría de esa información y degradaría su capacidad de detección.

De este modo, G se beneficia de los datos ya extraídos por C **sin renunciar** a su papel de salvaguarda: sigue actuando como barrera de cumplimiento **antes** de que se verifique la cobertura (D) y, sobre todo, **antes** de cualquier resolución o pago (E). Se concilia así la necesidad de datos del detector con la garantía de que ningún expediente fraudulento llega a pagarse.

5. Gestión de estado y ciclo de ejecución

El estado compartido del grafo se modela mediante `ClaimState`, un `TypedDict` que viaja entre nodos y constituye la única fuente de verdad durante la ejecución. LangGraph lo propaga automáticamente: cada agente lee lo que necesita y devuelve una actualización parcial que se fusiona con el estado existente. Sus campos se agrupan en cuatro bloques:

- **Identidad del expediente:** `claim_id`, `client_id`, `client_name`, `client_email`. El nombre del asegurado es necesario para el cribado OFAC de G.
- **Datos de entrada:** `claim_type`, `amount_requested`, `channel`, `documents` y `uploaded_files` (los archivos binarios reales para el análisis multimodal de C).
- **Resultados parciales** de cada agente: `validation_result` (B), `extraction_result` (C), `fraud_result` (G), `coverage_result` (D) y `resolution` (E).
- **Acumuladores y control de flujo:** `reasoning_trace`, `decisions_log`, `status`, `decision`, `hitl_required`, `terminate` y `termination_reason`.

Los **dos acumuladores** crecen a medida que avanza el flujo y son la base de la trazabilidad:

- **`reasoning_trace`** — la traza de Chain of Thought. Cada agente añade su razonamiento, de modo que al final se dispone de la narración completa del proceso de decisión.
- **`decisions_log`** — el registro de decisiones, con **una entrada por agente** (agente, acción, justificación, confianza, necesidad de HITL). Es la materialización auditable del proceso.

Ambos campos se declaran como `Annotated[list, operator.add]`, lo que indica a LangGraph que las contribuciones de los distintos nodos deben **acumularse** en lugar de sobrescribirse. Es un detalle técnico clave: permite que cada agente añada su entrada sin pisar las anteriores, preservando el Chain of Thought completo.

El ciclo de ejecución se gobierna desde la función `process_claim`, que actúa como punto de entrada de orquestación:

1. Se construye el `ClaimState` inicial a partir del expediente entrante.
2. Se ejecuta el grafo de LangGraph mediante `ainvoke`; cada nodo lee el estado, invoca su herramienta, razona y **escribe** su contribución en los acumuladores antes de ceder el control al supervisor.
3. Tras finalizar el grafo, la función auxiliar `_normalize_final_state` deduce el `status` y la `decision` finales en los casos en que el flujo se ha cortado sin pasar por el resolver (por ejemplo, fraude marcado por G o documentos incompletos detectados por B). Esto garantiza que la respuesta al cliente sea siempre coherente.
4. Finalmente, `process_claim` realiza la **persistencia centralizada** de todas las decisiones en MariaDB, en una única transacción.

Un aspecto esencial de resiliencia: la persistencia está **envuelta en try/except**. Si la base de datos no está disponible (por ejemplo, en la CLI de demostración sin MariaDB), la excepción se captura y el flujo **igualmente devuelve su resultado** (decisión + traza). Se trata de una persistencia *best-effort*: la demostración no depende de que MariaDB esté levantada, lo que refuerza la propiedad de resiliencia descrita en la sección 1.

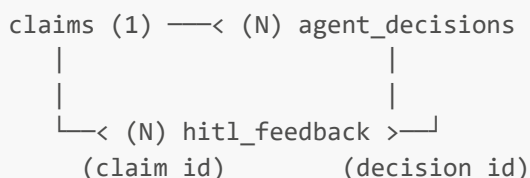
6. Datos y conocimiento

6.1. Modelo de datos (MariaDB)

La persistencia relacional se apoya en **MariaDB** mediante **SQLAlchemy 2.0 asíncrono** (driver `aiomysql`) y un patrón repositorio. El esquema se compone de tres tablas centrales, diseñadas para dar soporte a la trazabilidad y al ciclo HITL:

- **claims** — el expediente de reclamación. Recoge identidad del cliente, tipo de siniestro, canal, estado, importe solicitado e importe aprobado, con marcas temporales.
- **agent_decisions** — una fila por decisión de agente, con clave foránea a **claims**. Registra el agente, la acción, el razonamiento (CoT), la confianza, si requirió HITL y el momento. Es la materialización persistente del `decisions_log` y el soporte de la auditoría.
- **hitl_feedback** — registro de las anulaciones (*overrides*) humanas, con claves foráneas a **claims** y a **agent_decisions**. Conserva el revisor, la acción original automática, la acción final adoptada y la razón del *override*, permitiendo contrastar la decisión automática con la humana.

El campo `status` se modela como un **enum** con los valores `open`, `validating`, `extracting`, `checking_policy`, `checking_fraud`, `resolved`, `rejected`, `pending_review` y `closed`. Estos estados reflejan tanto las etapas del flujo (sección 4) como sus salidas terminales.



6.2. RAG sobre pólizas (ChromaDB)

La verificación de cobertura (Agente D) se apoya en **recuperación aumentada por generación (RAG)** (Lewis et al., 2020) sobre el corpus de pólizas. El enfoque RAG combina la recuperación de fragmentos relevantes desde una base de conocimiento con la generación de respuestas fundamentadas en esos fragmentos, lo que reduce las alucinaciones y permite **citar la cláusula concreta** que respalda una decisión de cobertura.

En esta entrega el RAG está **operativo: ChromaDB embebido** (en proceso) indexa los documentos de póliza (**data/policies/*.md**), cada uno con un *frontmatter* de metadatos (tipo de siniestro, sección, cobertura, límite, franquicia). El Agente D recupera la cláusula más relevante mediante **búsqueda vectorial con filtrado por el metadato claim_type**, robusta con corpus pequeño y escalable a varias cláusulas por tipo. Las pólizas son **sintéticas** (placeholder del prototipo), pendientes de sustitución por las condiciones reales de Seguros Pepín en un despliegue productivo.

7. Capacidades de IA reales frente a integraciones simuladas

La frontera entre lo real y lo simulado en este proyecto no separa "inteligencia artificial" de "reglas", sino **integraciones con terceros sin acceso** (necesariamente simuladas) de **capacidades de IA bajo control del proyecto** (reales). Esta sección formaliza ambas listas.

7.1. Integraciones externas simuladas (tabla *Mock* → *Integración real*)

Por cada integración con un sistema de Seguros Pepín al que no hay acceso, se indica el componente que la sustituiría en producción:

Integración simulada (<i>mock</i>)	Agente que la usa	Integración real en Seguros Pepín
Listas oficiales de sanciones (OFAC SDN / ONU)	G (fraude / cumplimiento)	Servicio corporativo de <i>screening</i> AML / sanciones con suscripción a las listas oficiales actualizadas.
Repositorio documental del expediente	B (validación documental)	Gestor documental / ECM corporativo de la aseguradora.
Catálogo y cláusulas de pólizas	D (verificación de cobertura)	Sistema <i>core</i> de pólizas (Policy Administration System), indexado por el RAG real.
Autorización y emisión de pagos	E (resolución)	Motor de pagos / tesorería de Seguros Pepín.
Canales de notificación al cliente	A / E (solicitud de información, resolución)	Plataformas reales de email, portal de cliente y WhatsApp Business.

7.2. Capacidades de IA reales

Frente a lo anterior, las siguientes capacidades se ejecutan de verdad, porque dependen de modelos y motores bajo control del proyecto (no de Seguros Pepín):

Capacidad real	Agente	Tecnología	Mecanismo de resiliencia
Extracción multimodal (visión)	C	Claude Vision (claude-sonnet-4-6)	<i>Fallback</i> a extracción simulada si no hay archivos o clave de API.
RAG sobre pólizas	D	ChromaDB embebido + búsqueda vectorial con filtro de metadato	<i>Fallback</i> determinista (check_policy) si el RAG no está disponible.
Motor de 4 detectores de fraude	G	Algoritmos deterministas (fuzzy matching, Z-score, ventana temporal, coherencia de fechas)	Determinista por diseño; no depende de servicios externos.

Capacidad real	Agente	Tecnología	Mecanismo de resiliencia
Razonamiento en lenguaje natural (CoT)	A–E, G	Claude (<code>claude-sonnet-4-6</code>) vía <code>reason()</code>	<i>Fallback</i> determinista equivalente si no hay clave de API.

La combinación de capacidades reales y *fallbacks* deterministas hace que el sistema sea, a la vez, **demostrable de verdad** (las capacidades de IA funcionan) y **resiliente** (el flujo se completa aunque falle cualquier componente opcional).

8. Human-in-the-Loop (HITL)

El sistema incorpora un mecanismo de **Human-in-the-Loop** que reserva a un revisor humano las decisiones de mayor riesgo, en lugar de automatizarlas ciegamente. El HITL se activa mediante un **dobles disparador**:

- Disparador por fraude / cumplimiento.** Cuando el Agente G emite veredicto `HIGH_RISK` o `BLOCKED`, el expediente se marca para revisión humana (o bloqueo) antes de continuar hacia la resolución. Es una salvaguarda de cumplimiento normativo.
- Disparador por importe.** Cuando el importe del siniestro supera el umbral configurable `HITL_AMOUNT_THRESHOLD` (5.000 € por defecto), la resolución (Agente E) se deriva a revisión humana aunque la cobertura sea correcta. Al ser configurable, el umbral permite ajustar el grado de automatización a la política de riesgo de la organización.

En ambos casos el expediente queda en estado `pending_review`. Las decisiones revisadas se registran en la tabla `hitl_feedback`, que conserva la acción original automática, la acción final adoptada y la razón del *override*. Este registro no solo cierra el bucle de auditoría, sino que constituye una fuente de datos para una eventual mejora futura de los criterios de decisión automáticos.

Conviene precisar la diferencia de naturaleza entre los dos disparadores. La **revisión por importe** responde a una decisión de política discrecional: la organización elige a partir de qué cuantía prefiere supervisión humana, y ese umbral es configurable. La **coincidencia con listas de sanciones OFAC/ONU**, en cambio, constituye una obligación legal vinculante sin margen de discrecionalidad; por ello el sistema la trata como bloqueo automático (`RECHAZO_FRAUDE`) en lugar de derivarla a revisión. Como refinamiento, dicho bloqueo podría redirigirse a un oficial de cumplimiento (HITL de cumplimiento) para su confirmación formal, manteniendo el principio de intervención humana en los casos sensibles y creando además una traza documentada del proceso.

9. Despliegue y stack tecnológico

9.1. Despliegue

El sistema admite **dos modos de despliegue** complementarios.

(1) Docker Compose — arquitectura completa. Empaqueta el sistema en **cinco servicios**, con base de datos y servicios reales:

Servicio	Función	Puerto
<code>backend</code>	API REST (FastAPI + Uvicorn) y orquestación de agentes	8000
<code>frontend</code>	Interfaz de demostración (Streamlit)	8501
<code>chromadb</code>	Base vectorial para el RAG de pólizas	8080
<code>mariadb</code>	Base de datos relacional	3306
<code>adminer</code>	Administración web de la base de datos	8082

(2) App Streamlit autónoma — demo en vivo. El fichero `streamlit_app.py` es una aplicación autónoma desplegable en **Streamlit Community Cloud** que ejecuta los agentes **en proceso**, sin backend ni MariaDB. Pensada para la demostración en vivo, aprovecha la persistencia *best-effort* (sección 5) y los *fallbacks* deterministas (sección 7) para funcionar sin la infraestructura completa.

9.2. Stack tecnológico

Componente	Tecnología
Lenguaje	Python 3.11
API REST	FastAPI + Uvicorn
Orquestación agéntica	LangGraph + LangChain (Chase, 2022; LangChain AI, 2024)
LLM (visión y razonamiento)	Claude Sonnet 4.6 (<code>claude-sonnet-4-6</code>) vía API de Anthropic (Anthropic, 2024)
Base vectorial (RAG)	ChromaDB (embebido en la app autónoma; servicio en Docker)
Base de datos relacional	MariaDB 11.3
Acceso a datos	SQLAlchemy 2.0 async (driver aiomysql); SQLite en memoria para tests
Frontend / demo	Streamlit
Empaquetado / despliegue	Docker Compose (5 servicios) + Streamlit Community Cloud
Calidad	100 tests automatizados (pytest) sobre SQLite en memoria, sin dependencia de MariaDB

En cuanto a la **calidad**, el proyecto cuenta con **100 pruebas automatizadas** ejecutadas con pytest sobre una base de datos SQLite en memoria, que cubren los agentes individuales, el flujo de orquestación completo, los detectores de fraude (incluida la coherencia documental), el RAG, la capa de repositorio, el helper de razonamiento, los endpoints de la API REST, el blindaje de entrada, el asistente de conciliación (Agente F) y la interfaz Streamlit. Al no depender de MariaDB, la suite es reproducible en cualquier entorno. El registro completo de pruebas y resultados está en `docs/testing/registro-tests.md`.

Nota sobre localización. Los identificadores de tipo de siniestro (`danys_propis`, `responsabilitat`, `robatori`, `danys_mecanics`) y la moneda de referencia (euros) se heredan del andamiaje inicial del prototipo. En una implantación real para Seguros Pepín (República Dominicana) se localizarían a castellano dominicano y a pesos dominicanos (DOP / RD\$); las etiquetas visibles para el usuario ya se presentan en castellano. Esta adaptación afectaría únicamente a los valores de las enumeraciones internas y a la capa de presentación, sin alterar la lógica de los agentes.

10. Bibliografía

Anthropic. (2024). *Tool use (function calling) — Claude API documentation*. <https://docs.anthropic.com/en/docs/tool-use>

Chase, H. (2022). *LangChain* [Software]. <https://github.com/langchain-ai/langchain>

LangChain AI. (2024). *LangGraph documentation: Multi-agent supervisor pattern*. https://langchain-ai.github.io/langgraph/tutorials/multi_agent/agent_supervisor/

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... Wen, J. R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345. <https://doi.org/10.1007/s11704-024-40231-1>

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models*. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>

ewpage

Capítulo 2 — Herramientas y capacidades

3. Herramientas y capacidades del sistema agéntico

3.1 Introducción: el papel de las herramientas en un sistema agéntico

En los sistemas de inteligencia artificial basados en agentes, el término *herramienta* (*tool*) designa una función de código que el modelo de lenguaje puede invocar de forma autónoma cuando lo considera necesario para resolver una tarea. Esta capacidad, conocida como *function calling* o *tool use*, transforma al modelo de lenguaje de un generador de texto en un agente capaz de actuar sobre el entorno: consultar bases de datos, ejecutar cálculos, analizar documentos o emitir notificaciones (Anthropic, 2024). El mecanismo es el siguiente: junto con el mensaje del usuario, se expone al modelo una descripción estructurada de cada herramienta disponible —nombre, parámetros y propósito—; el modelo razona sobre cuál invocar y con qué argumentos; el orquestador ejecuta la función y devuelve el resultado para que el modelo continúe su razonamiento. Este ciclo de *razonamiento* → *acción* → *observación* sustenta el paradigma ReAct (Yao et al., 2023), que inspira el helper `reason()` empleado en Smart-Claims Agent.

En la implementación concreta del proyecto, las herramientas se definen mediante el decorador `@tool` de LangChain (Chase, 2022), que analiza la firma de la función Python y su *docstring* para construir automáticamente el esquema JSON que se expone al modelo. El código reside principalmente en `backend/app/tools/claim_tools.py` y `backend/app/tools/fraud_tools.py`, y las funciones son invocadas desde los nodos de los agentes del grafo LangGraph.

3.1.1 Distinción fundamental: mocks externos frente a capacidades de IA reales

Un aspecto crítico del diseño del prototipo, y que debe quedar explícitamente claro, es que existen **dos categorías conceptualmente distintas** de herramientas y capacidades en el sistema:

Categoría 1 — Mocks de sistemas externos de Seguros Pepín. El prototipo Smart-Claims Agent se ha desarrollado sin acceso a los sistemas reales de la empresa. La gestión documental corporativa, el core asegurador, la pasarela de pagos, el CRM y el portal del cliente son sistemas propietarios de Seguros Pepín, S.A. a los que el proyecto académico no tiene conexión. En consecuencia, las herramientas `@tool` que representan estas integraciones son *mocks definitivos*: implementaciones simuladas que reproducen fielmente la interfaz (firma, esquema de entrada y salida) que tendría cada integración en producción, pero cuya lógica interna genera datos sintéticos o deterministas. Estas herramientas son `validate_documents`, `extract_multimodal` (en su rol de fallback), `check_policy` (en su rol de fallback), `approve_payment`, `send_rejection` y `request_more_info`.

Categoría 2 — Capacidades de IA reales. Claude no es un sistema de Seguros Pepín: es el LLM propio del proyecto. Tampoco lo son ChromaDB ni el motor antifraude. Por tanto, las capacidades construidas sobre estas tecnologías se

implementan de forma real y funcional en el prototipo: la extracción multimodal con Claude Vision ([backend/app/agents/vision.py](#)), el motor antifraude de cuatro detectores deterministas ([backend/app/tools/fraud_tools.py](#)) y la base de conocimiento RAG de pólizas basada en ChromaDB ([backend/app/rag/policy_store.py](#)) son implementaciones reales, no simuladas, que operan en el sistema durante la ejecución.

Esta distinción determina la arquitectura de resiliencia del sistema: cuando la capacidad real no está disponible (sin clave de API, sin ChromaDB inicializado), el sistema cae de forma controlada al mock correspondiente, garantizando que la demostración nunca se interrumpe.

3.2 Tabla resumen de herramientas y capacidades

La tabla siguiente ofrece una visión consolidada del catálogo completo del sistema.

#	Herramienta / Capacidad	Agente principal	Categoría	Propósito
1	validate_documents	B — Validación documental	Mock externo	Verificar documentación aportada y vigencia de la póliza
2	extract_multimodal	C — Extracción multimodal (fallback)	Mock externo	Extracción simulada de datos de documentos (ruta de respaldo)
3	Claude Vision (analyze_document)	C — Extracción multimodal (ruta real)	IA real	Extracción estructurada de imágenes y PDF con Claude Vision
4	check_policy	D — Verificación de cobertura (fallback)	Mock externo	Consulta determinista de cobertura y franquicia
5	RAG de pólizas (retrieve_policy)	D — Verificación de cobertura (ruta real)	IA real	Recuperación semántica de cláusulas de póliza con ChromaDB
6	Motor antifraude (4 detectores)	G — Fraude y cumplimiento	IA real	Cribado antifraude determinista con scoring compuesto
7	approve_payment	E — Resolución	Mock externo	Emisión simulada de orden de pago al asegurado
8	send_rejection	E — Resolución	Mock externo	Comunicación simulada de resolución denegatoria
9	request_more_info	B — Validación documental	Mock externo	Solicitud simulada de documentación adicional al cliente

Adicionalmente, [reason\(\)](#) ([backend/app/agents/reasoning.py](#)) y el repositorio ([backend/app/db/repository.py](#)) constituyen módulos de apoyo transversales, no herramientas en el sentido estricto del *function calling*, y se documentan en la sección 3.6.

3.3 Herramientas [@tool](#) de LangChain (mocks de sistemas externos)

Las siguientes subsecciones describen cada herramienta definida con el decorador [@tool](#) en [backend/app/tools/claim_tools.py](#). Todas ellas simulan la interfaz de sistemas externos de Seguros Pepín que no están disponibles en el entorno de desarrollo. La estructura de cada subsección cubre: propósito, parámetros de entrada, esquema de salida, agente que la invoca y la proyección de integración real.

3.3.1 [validate_documents](#)

Propósito. Verifica que el expediente de reclamación contiene la documentación mínima exigida para el tipo de siniestro declarado y que la póliza asociada se encuentra en vigor. El conjunto de documentos requeridos está centralizado en la constante `REQUIRED_DOCS_BY_TYPE`, compartida entre la herramienta y el Agente B para garantizar una única fuente de verdad.

Los tipos documentales requeridos por tipo de siniestro son:

Tipo de siniestro	Documentos requeridos
<code>danys_propis</code>	<code>foto_danys</code> , <code>factura</code> , <code>denuncia_companyia</code>
<code>responsabilitat</code>	<code>foto_danys</code> , <code>acta_policial</code> , <code>dades_tercer</code>
<code>robatori</code>	<code>acta_policial</code> , <code>llista_objectes_robats</code>
<code>danys_mecanics</code>	<code>informe_taller</code> , <code>factura</code>
<code>default</code>	<code>foto_danys</code> , <code>factura</code>

Parámetros de entrada.

Parámetro	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador único del expediente
<code>claim_type</code>	<code>str</code>	Tipo de siniestro (<code>danys_propis</code> , <code>responsabilitat</code> , <code>robatori</code> , <code>danys_mecanics</code> , <code>default</code>)
<code>doc_types</code>	<code>list[str]</code>	Lista de tipos documentales aportados por el cliente

Esquema de salida.

Campo	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador del expediente
<code>claim_type</code>	<code>str</code>	Tipo de siniestro evaluado
<code>is_valid</code>	<code>bool</code>	Indica si el conjunto documental es suficiente
<code>missing_docs</code>	<code>list[str]</code>	Tipos documentales ausentes
<code>required_docs</code>	<code>list[str]</code>	Lista completa de documentos exigidos
<code>provided_docs</code>	<code>list[str]</code>	Documentos efectivamente aportados
<code>contract_active</code>	<code>bool</code>	Estado de la póliza (siempre <code>True</code> en el mock)
<code>checked_at</code>	<code>str</code>	Marca temporal ISO 8601 de la verificación

Agente que la invoca. Nodo B (Validación documental). Si `is_valid` es `False`, el Agente B activa también `request_more_info` para solicitar los documentos faltantes al cliente.

Mock → Integración real. En producción, esta herramienta debería dirigirse al **gestor documental corporativo (ECM)** de Seguros Pepín para confirmar la presencia e integridad de los ficheros adjuntos, y al **core asegurador** (sistema de gestión de pólizas) para validar que el contrato estaba activo en la fecha del siniestro declarado. El campo `contract_active` siempre devuelve `True` en el mock, lo que constituye la simplificación más relevante de esta herramienta.

3.3.2 `extract_multimodal` (fallback del Agente C)

Propósito. Esta herramienta actúa como **ruta de respaldo** del Agente C cuando la extracción real mediante Claude Vision no está disponible. Devuelve datos sintéticos plausibles para cada tipo documental reconocido, con una puntuación de confianza generada aleatoriamente en el rango [0,82 – 0,98]. Es importante subrayar que esta herramienta no procesa ningún fichero real: la extracción genuina de imágenes y documentos la realiza la función `analyze_document()` de `backend/app/agents/vision.py` (véase la sección 3.4).

Parámetros de entrada.

Parámetro	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador del expediente
<code>file_url</code>	<code>str</code>	URL o ruta del fichero a analizar (no se procesa en el mock)
<code>doc_type</code>	<code>str</code>	Tipo documental (<code>foto_danys</code> , <code>factura</code> , <code>acta_policial</code> , etc.)

Esquema de salida.

Campo	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador del expediente
<code>doc_type</code>	<code>str</code>	Tipo documental procesado
<code>extracted</code>	<code>dict</code>	Datos sintéticos estructurados según el tipo documental
<code>confidence</code>	<code>float</code>	Confianza simulada en [0,82; 0,98]
<code>model</code>	<code>str</code>	Identificador " <code>claude-sonnet-4-6 (mock)</code> "
<code>extracted_at</code>	<code>str</code>	Marca temporal de la extracción (ISO 8601)

Agente que la invoca. Nodo C (Extracción multimodal), únicamente como fallback cuando `analyze_document()` devuelve `None`.

Mock → Integración real. La ruta real ya existe en el sistema: la función `analyze_document()` invoca a Claude Vision con los ficheros reales del expediente (véase la sección 3.4.1). En producción, la ruta de fallback podría complementarse con un motor de OCR clásico (por ejemplo, Tesseract) para documentos de baja resolución que la VLM no logre interpretar con suficiente confianza.

Nota sobre `check_fraud`. El módulo `claim_tools.py` contiene también una función `check_fraud` registrada como `@tool`, que genera un score de riesgo aleatorio en el rango [0,01 – 0,35]. Esta función existía en una versión anterior del sistema, pero el **Agente G ya no la utiliza**: el Agente G emplea exclusivamente el motor antifraude real de cuatro detectores de `fraud_tools.py` (sección 3.5). La función `check_fraud` se mantiene en el módulo por compatibilidad de interfaz, pero no es invocada en el flujo de producción y no debe confundirse con el motor antifraude real.

3.3.3 `check_policy` (fallback del Agente D)

Propósito. Determina si el tipo de siniestro declarado está cubierto por la póliza del asegurado y calcula el importe neto a abonar tras aplicar el límite de cobertura y la franquicia. Esta herramienta actúa como **ruta de respaldo** del Agente D cuando el RAG de pólizas no está disponible. Los parámetros de cobertura están codificados de forma estática en la constante `coverage_rules`, que replica los valores del condicionado de Seguros Pepín con fines de prototipado.

Tipo de siniestro	Cubierto	Límite máximo	Franquicia	Sección
<code>danys_propis</code>	Sí	10 000 €	300 €	SP-PCS-009 § 3.2
<code>responsabilitat</code>	Sí	50 000 €	0 €	SP-PCS-009 § 4.1
<code>robatori</code>	Sí	8 000 €	500 €	SP-PCS-009 § 5.0
<code>danys_mecanics</code>	No	—	—	SP-PCS-009 § 7.3 (exclusión)

Parámetros de entrada.

Parámetro	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador del expediente
<code>claim_type</code>	<code>str</code>	Tipo de siniestro
<code>amount</code>	<code>float</code>	Importe reclamado en euros

Esquema de salida.

Campo	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador del expediente
<code>claim_type</code>	<code>str</code>	Tipo de siniestro evaluado
<code>amount_requested</code>	<code>float</code>	Importe reclamado
<code>covered</code>	<code>bool</code>	Indica si el siniestro está amparado
<code>max_coverage</code>	<code>float</code>	Límite máximo de cobertura (€)
<code>deductible</code>	<code>float</code>	Franquicia a cargo del asegurado (€)
<code>net_payable</code>	<code>float</code>	Importe neto a satisfacer: $\max(0, \min(\text{amount}, \text{max_coverage}) - \text{deductible})$
<code>policy_section</code>	<code>str</code>	Cláusula o sección de la póliza que ampara la decisión

Agente que la invoca. Nodo D (Verificación de cobertura), únicamente cuando `retrieve_policy()` del RAG devuelve `None`.

Mock → Integración real. La ruta real ya existe en el sistema: `retrieve_policy()` realiza una búsqueda vectorial sobre las pólizas indexadas en ChromaDB (sección 3.4.2). En producción, el corpus se alimentaría con los condicionados reales de Seguros Pepín en lugar de los documentos sintéticos de prototipado.

3.3.4 `approve_payment`

Propósito. Emite la orden de pago al asegurado cuando la reclamación ha sido aprobada. Simula la integración con la pasarela de pagos o el core financiero de Seguros Pepín generando un identificador de transacción aleatorio y una fecha de abono programada.

Parámetros de entrada.

Parámetro	Tipo	Descripción
claim_id	str	Identificador del expediente
amount	float	Importe a abonar (€)
iban	str	Número de cuenta del beneficiario

Esquema de salida.

Campo	Tipo	Descripción
claim_id	str	Identificador del expediente
transaction_id	str	Identificador único de transacción (formato TXN-{claim_id}-{randint})
amount	float	Importe abonado (€)
iban_last4	str	Últimos cuatro dígitos del IBAN (trazabilidad sin exponer datos sensibles)
status	str	Estado de la orden ("scheduled")
scheduled_date	str	Fecha prevista de transferencia (estática en el mock)

Agente que la invoca. Nodo E (Resolución), cuando el agente determina que la reclamación es válida, está cubierta y el importe está por debajo del umbral de revisión humana (HITL).

Mock → Integración real. En producción, esta herramienta se conectaría a la **pasarela de pagos o al módulo financiero** del core asegurador de Seguros Pepín para emitir una transferencia bancaria real, incluyendo los controles de autorización, firma y reconciliación que exige la operativa aseguradora. El campo `scheduled_date` estático pasaría a calcularse dinámicamente según los plazos regulatorios aplicables.

3.3.5 `send_rejection`

Propósito. Comunica al asegurado la resolución denegatoria de su reclamación, incluyendo una justificación generada por el Agente E y los plazos para ejercer el derecho de reclamación, en cumplimiento con las obligaciones de información de la normativa de seguros.

Parámetros de entrada.

Parámetro	Tipo	Descripción
claim_id	str	Identificador del expediente
reason	str	Motivo del rechazo, generado por el LLM del Agente E
client_email	str	Dirección de correo electrónico del asegurado

Esquema de salida.

Campo	Tipo	Descripción
claim_id	str	Identificador del expediente
email_id	str	Identificador del mensaje (formato EMAIL-{claim_id}-REJ)
sent_to	str	Dirección de destino

Campo	Tipo	Descripción
<code>reason_summary</code>	<code>str</code>	Primeros 200 caracteres del motivo de rechazo
<code>sent_at</code>	<code>str</code>	Marca temporal del envío (ISO 8601)

Agente que la invoca. Nodo E (Resolución), cuando el Agente D ha determinado que el siniestro no está cubierto por la póliza.

Mock → Integración real. En producción, la herramienta invocaría el **sistema de notificaciones corporativo o el CRM** de Seguros Pepín para generar y enviar la comunicación por el canal acordado con el cliente (correo electrónico, SMS, área de cliente), con registro de acuse de recibo para fines de cumplimiento normativo.

3.3.6 `request_more_info`

Propósito. Solicita al asegurado que aporte la documentación o información adicional necesaria para continuar con la tramitación, especificando los campos concretos que faltan y el plazo disponible para su presentación (diez días por defecto en el mock).

Parámetros de entrada.

Parámetro	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador del expediente
<code>missing_fields</code>	<code>list[str]</code>	Lista de campos o tipos documentales ausentes
<code>client_email</code>	<code>str</code>	Dirección de correo electrónico del asegurado

Esquema de salida.

Campo	Tipo	Descripción
<code>claim_id</code>	<code>str</code>	Identificador del expediente
<code>request_id</code>	<code>str</code>	Identificador de la solicitud (formato <code>INFO-{claim_id}-{randint}</code>)
<code>fields_requested</code>	<code>list[str]</code>	Campos solicitados (eco de la entrada)
<code>sent_to</code>	<code>str</code>	Dirección de destino
<code>deadline_days</code>	<code>int</code>	Días concedidos al asegurado para responder (10)
<code>sent_at</code>	<code>str</code>	Marca temporal del envío (ISO 8601)

Agente que la invoca. Nodo B (Validación documental), cuando `validate_documents` detecta que faltan documentos requeridos.

Mock → Integración real. En producción, la herramienta interactuaría con el **portal del cliente de Seguros Pepín** o con el proveedor de correo electrónico transaccional para generar una comunicación personalizada con enlace directo a la sección de carga de documentos, e integraría el seguimiento del estado en el sistema de gestión de expedientes.

3.4 Capacidades de IA reales

A diferencia de las herramientas de la sección anterior, las capacidades descritas en esta sección están implementadas de forma real y funcional en el prototipo. Son parte del núcleo tecnológico del proyecto y no simulaciones de sistemas externos.

3.4.1 Extracción multimodal con Claude Vision ([backend/app/agents/vision.py](#))

El Agente C dispone de una capacidad de extracción documental real construida sobre las capacidades de visión del modelo `claude-sonnet-4-6`. La función `analyze_document(data: bytes, media_type: str, filename: str)` recibe el contenido binario de un fichero adjunto, lo codifica en Base64 y lo envía a la API de Claude junto con un *prompt* de extracción estructurado, obteniendo como respuesta un objeto JSON con los campos del expediente.

Tipos de documento soportados. La función acepta imágenes en formato PNG, JPEG y WebP, así como documentos PDF. El tipo MIME determina si el bloque multimedia se construye como `"image"` o `"document"` en la solicitud a la API:

```
if media_type == "application/pdf":
    return {"type": "document",
            "source": {"type": "base64", "media_type": "application/pdf", "data": b64}}
return {"type": "image",
        "source": {"type": "base64", "media_type": media_type, "data": b64}}
```

Esquema de salida. Claude devuelve un JSON plano que se valida y se enriquece con el nombre del fichero y el identificador del modelo:

Campo	Tipo	Descripción
<code>doc_type</code>	<code>str</code>	Tipo documental reconocido (<code>factura</code> , <code>foto_danos</code> , <code>acta</code> , <code>informe_taller</code> , <code>otro</code>)
<code>amount</code>	<code>float</code> <code>null</code>	Importe en euros detectado, o <code>null</code> si no aplica
<code>date</code>	<code>str</code> <code>null</code>	Fecha en formato YYYY-MM-DD, o <code>null</code>
<code>vendor</code>	<code>str</code> <code>null</code>	Emisor, taller o entidad identificada, o <code>null</code>
<code>summary</code>	<code>str</code>	Resumen breve en castellano del contenido del documento
<code>confidence</code>	<code>float</code>	Confianza de la extracción entre 0 y 1, autoevaluada por el modelo
<code>filename</code>	<code>str</code>	Nombre del fichero analizado
<code>model</code>	<code>str</code>	Identificador del modelo empleado (" <code>claude-sonnet-4-6</code> ")

Lógica de degradación controlada. Si la variable de entorno `ANTHROPIC_API_KEY` no está configurada, o si la llamada a la API falla por cualquier motivo (red, cuota, error de parseo del JSON de respuesta), la función devuelve `None`. El Agente C detecta este valor y cae al mock `extract_multimodal`, garantizando que la demostración no se interrumpe. Esta política de *graceful degradation* es consistente con el resto del sistema.

Relevancia académica. Esta implementación constituye el ejemplo más directo del uso de modelos multimodales en el contexto asegurador: permite que el sistema analice automáticamente fotografías de daños, facturas digitalizadas y actas policiales, reduciendo la intervención manual en la fase de instrucción del expediente. La capacidad de visión de los LLM modernos para extraer información estructurada de documentos no estructurados es uno de los avances más significativos de los últimos años en el campo de la automatización de procesos empresariales (Anthropic, 2024).

3.4.2 RAG de pólizas con ChromaDB (backend/app/rag/policy_store.py)

El Agente D puede consultar las condiciones de cobertura de las pólizas mediante un componente de *Retrieval-Augmented Generation* (RAG; Lewis et al., 2020) construido sobre ChromaDB embebido en proceso. Este enfoque permite que el agente recupere la cláusula más relevante para un tipo de siniestro concreto y cite la sección de la póliza que fundamenta su decisión, en lugar de aplicar reglas estáticas.

Arquitectura del componente. El módulo `policy_store.py` expone dos funciones principales:

- `_build_collection()`: indexa los documentos de póliza en formato Markdown (con *frontmatter* YAML) alojados en `data/policies/*.md` en una colección ChromaDB embebida en memoria. Cada póliza se representa mediante un texto de indexación conciso anclado en el tipo de siniestro, enriquecido con los metadatos estructurados extraídos del *frontmatter* (`claim_type`, `section`, `covered`, `max_coverage`, `deductible`).
- `retrieve_policy(claim_type: str, description: str)`: realiza una búsqueda vectorial filtrando por el metadato `claim_type` y devuelve la cláusula más relevante. El filtrado combinado (búsqueda semántica + filtro de metadato exacto) garantiza que el resultado corresponde siempre al tipo de siniestro correcto, incluso cuando el corpus crece con múltiples cláusulas por tipo.

Esquema de salida de `retrieve_policy`.

Campo	Tipo	Descripción
<code>claim_type</code>	<code>str</code>	Tipo de siniestro de la cláusula recuperada
<code>section</code>	<code>str</code>	Referencia a la sección de la póliza (p. ej., <code>SP-PCS-009 § 3.2</code>)
<code>covered</code>	<code>bool</code>	Indica si el siniestro está cubierto según la cláusula recuperada
<code>max_coverage</code>	<code>float</code>	Límite máximo de cobertura en euros
<code>deductible</code>	<code>float</code>	Franquicia aplicable en euros
<code>snippet</code>	<code>str</code>	Fragmento textual de la cláusula recuperada (hasta 280 caracteres)
<code>distance</code>	<code>float null</code>	Distancia vectorial entre la consulta y el documento recuperado

Control de disponibilidad. El componente está gestionado por la variable de entorno `SCA_RAG_ENABLED`. Si ChromaDB no está instalado, si el directorio `data/policies/` no existe o si la indexación falla, el módulo registra el fallo en el log y devuelve `None`, activando el fallback determinista `check_policy`. El indicador interno `_load_failed` evita reintentos innecesarios en bucle.

Naturaleza de las pólizas en el prototipo. Los documentos indexados son pólizas sintéticas creadas para el prototipo, con estructura de condicionados verosímil y referencias a las secciones del procedimiento SP-PCS-009. En un entorno de producción, estos documentos se sustituirían por los condicionados reales de Seguros Pepín, lo que no requeriría ningún cambio en el código del componente RAG, únicamente en los datos de entrada.

Fundamento teórico. El paradigma RAG, introducido por Lewis et al. (2020), combina la capacidad generativa de los LLM con la precisión de los sistemas de recuperación de información, permitiendo que el modelo base sus respuestas en documentos concretos y recuperables en lugar de en conocimiento paramétrico potencialmente desactualizado o incorrecto. En el contexto de la verificación de cobertura aseguradora, esta propiedad es especialmente valiosa: la decisión del Agente D queda fundamentada en una cláusula concreta de la póliza, con trazabilidad directa hacia el documento fuente.

3.4.3 Motor antifraude de cuatro detectores (`backend/app/tools/fraud_tools.py`)

El Agente G implementa un sistema modular de detección de fraude compuesto por cuatro detectores deterministas y una función de scoring compuesto. A diferencia de enfoques de caja negra, este diseño permite atribuir cada señal de riesgo a una causa concreta y auditable. Los cuatro detectores y la función de scoring residen en `backend/app/tools/fraud_tools.py` y constituyen una implementación real, no un mock.

Detector 1 — Verificación OFAC/ONU (`check_ofac_sanctions`)

Verifica el nombre del cliente declarante contra una lista de sanciones financieras que simula la SDN (*Specially Designated Nationals*) de la Oficina de Control de Activos Extranjeros (OFAC) del Departamento del Tesoro de los Estados Unidos y la lista consolidada de sanciones de las Naciones Unidas.

La lista interna `_SANCTIONS_LIST` contiene **15 entidades y personas físicas** sintéticas (ocho con lista SDN, siete con lista ONU) con nombres plausibles que permiten probar la detección tanto de personas físicas como jurídicas.

El algoritmo emplea *fuzzy matching* mediante `difflib.SequenceMatcher` con normalización previa de caracteres acentuados (conversión NFD + eliminación de diacríticos mediante `unicodedata`) para garantizar que variantes ortográficas como "Amira Belhaj" y "Amira Belhàj" produzcan similitudes equivalentes. El umbral de detección está fijado en **0,82**: una similitud igual o superior a este valor se considera coincidencia positiva y desencadena un veredicto `BLOCKED`.

La función devuelve un `NamedTuple` tipado (`OFACResult`) con los campos: `matched`, `entity_id`, `entity_name`, `similarity` y `sanction_list`.

Detector 2 — Anomalía de importe (`check_amount_anomaly`)

Detecta importes estadísticamente anómalos comparando el importe reclamado con los baselines históricos por tipo de siniestro mediante la métrica **Z-score**:

$$Z = \frac{x - \mu}{\sigma}$$

donde x es el importe reclamado, μ la media histórica y σ la desviación típica para el tipo de siniestro. Los baselines de la constante `_AMOUNT_BASELINES` son:

Tipo de siniestro	Media (€)	Desv. típica (€)	Máximo legítimo (€)
<code>danys_propis</code>	2 800	1 400	9 000
<code>responsabilitat</code>	12 000	8 000	48 000
<code>robatori</code>	3 200	1 600	7 500
<code>danys_mecanics</code>	800	400	3 000
<code>_default</code>	3 000	2 000	10 000

Un importe se considera anómalo si $|Z| > 2,0$ (umbral `_ZSCORE_THRESHOLD`) o si supera el máximo legítimo definido para su tipo. Devuelve `AmountResult` con: `flagged`, `z_score`, `requested`, `mean`, `std` y `exceeded_max`.

Detector 3 — Duplicados recientes (`check_duplicate_claims`)

Identifica reclamaciones del mismo cliente y tipo de siniestro dentro de una ventana temporal configurable (90 días por defecto). El detector recibe el historial de reclamaciones previas como parámetro (`existing_claims`:

`list[dict]`)), lo que permite pruebas unitarias deterministas sin acceso a la base de datos. En la integración real, este historial se obtendría mediante consulta asíncrona a MariaDB con índice compuesto por (`client_id`, `claim_type`, `created_at`).

Devuelve `DuplicateResult` con: `found`, `matching_claim_ids` y `days_since_last`.

Detector 4 — Coherencia documental (`check_document_coherence`)

Analiza las fechas presentes en los datos extraídos de los documentos del expediente buscando inconsistencias temporales. Las comprobaciones implementadas son:

- **Fecha de siniestro futura:** `incident_date > ahora`.
- **Fecha de siniestro excesivamente antigua:** `incident_date < 2015-01-01`.
- **Reclamación previa al siniestro:** `claim_date < incident_date`.
- **Factura anterior al siniestro:** `factura_date < incident_date - 30 días`.

La función soporta múltiples formatos de fecha mediante un parser secuencial (`%Y-%m-%d`, `%d/%m/%Y`, `%Y-%m-%dT%H:%M:%S`, `%Y-%m-%dT%H:%M:%S.%f`). Devuelve `DocCoherenceResult` con: `incoherent` y `issues` (lista de cadenas descriptivas de cada inconsistencia detectada).

Scoring compuesto (`compute_risk_score`)

La función `compute_risk_score(ofac, amount, duplicate, doc)` combina los cuatro resultados con pesos calibrados y emite uno de cuatro **veredictos graduados**:

Fuente de riesgo	Contribución máxima al score
OFAC/ONU: coincidencia confirmada	<code>BLOCKED</code> inmediato (score = 1,0)
Importe: supera el máximo legítimo	+0,40
Importe: Z-score anómalo (sin superar máximo)	hasta +0,35 (proporcional a <code> Z </code>)
Duplicados recientes (< 30 días)	+0,35
Duplicados recientes (≥ 30 días)	+0,23
Incoherencia documental	+0,10 por issue detectada (máximo +0,25)

Los veredictos resultantes son:

Veredicto	Condición	Consecuencia en el flujo
<code>BLOCKED</code>	Coincidencia OFAC/ONU confirmada	Rechazo automático, flujo terminado
<code>HIGH_RISK</code>	Score ≥ 0,55	HITL obligatorio; el supervisor detiene el flujo
<code>MEDIUM_RISK</code>	Score ≥ 0,25	HITL recomendado; decisión humana
<code>CLEAR</code>	Score < 0,25	Flujo continúa al Agente D

El campo `is_flagged` del estado del expediente se activa cuando el veredicto es `HIGH_RISK` o `BLOCKED`.

Mock → Integración real. En producción, cada detector conectaría con su fuente de datos real:

Detector	Fuente real proyectada
OFAC/ONU	Descarga periódica del fichero SDN oficial de OFAC + lista ONU consolidada de la ONU
Anomalía de importe	Cálculo dinámico de μ y σ sobre la tabla <code>claims</code> histórica de Seguros Pepín
Duplicados	Consulta asíncrona a MariaDB con índice por <code>(client_id, claim_type, created_at)</code>
Coherencia documental	Datos extraídos por el Agente C mediante Claude Vision (sección 3.4.1)

3.5 Estrategia de simulación: mocks definitivos

La decisión de implementar las herramientas de sistemas externos como mocks definitivos —en lugar de mocks temporales o integraciones parciales— responde a cuatro criterios fundamentales:

Reproducibilidad. Un sistema agéntico presenta comportamiento no determinista a nivel del razonamiento del LLM, pero el entorno de pruebas debe ser reproducible para validar los caminos de ejecución del grafo. Al controlar los valores de salida de las herramientas, es posible verificar sistemáticamente que cada nodo produce la respuesta esperada ante cada escenario de prueba.

Independencia de sistemas externos. El desarrollo del prototipo no puede estar condicionado por la disponibilidad, los tiempos de respuesta o los costes de acceso a los sistemas reales de Seguros Pepín. Los mocks definitivos eliminan esta dependencia y permiten iterar con rapidez durante la fase de investigación.

Fidelidad de interfaz. Aunque la lógica interna es simulada, los esquemas de entrada y salida de cada herramienta son idénticos a los que tendría la integración real. Esto garantiza que la sustitución de un mock por su equivalente productivo sea un cambio de implementación interna, sin necesidad de modificar el código de los agentes ni la estructura del grafo.

Cobertura de escenarios. El conjunto de mocks cubre todos los caminos de decisión del grafo: reclamación válida y pagada, reclamación rechazada por falta de cobertura, reclamación suspendida por solicitud de documentación, reclamación bloqueada por alerta antifraude y reclamación derivada a revisión humana por importe. Esta cobertura completa es esencial para la evaluación del sistema en el contexto del TFM.

3.6 Módulos de apoyo transversales

Además de las herramientas y capacidades descritas, el sistema cuenta con dos módulos de apoyo que no son herramientas en el sentido del *function calling*, pero que intervienen de forma transversal en todos los agentes.

3.6.1 Helper de razonamiento con CoT (`backend/app/agents/reasoning.py`)

La función `reason(system: str, prompt: str, fallback: str) -> str` encapsula la generación de razonamiento en lenguaje natural mediante Claude. Si la variable de entorno `ANTHROPIC_API_KEY` está disponible, invoca al modelo `claude-sonnet-4-6` a través de `ChatAnthropic` con temperatura 0 para maximizar la consistencia. Si la clave no está configurada o si la llamada falla por cualquier motivo, devuelve el argumento `fallback` —un texto determinista proporcionado por el agente que llama—. Esta dualidad garantiza que la demostración funcione siempre, independientemente de la conectividad, y reduce drásticamente el coste de los tests de integración, que no necesitan emular la red.

3.6.2 Repositorio de persistencia (`backend/app/db/repository.py`)

El repositorio centraliza todo el acceso a la base de datos relacional (MariaDB en producción, SQLite en memoria en tests). Los agentes **no acceden directamente a la base de datos**: acumulan sus contribuciones en el campo `decisions_log` del estado compartido del grafo LangGraph durante la ejecución, y la función `process_claim` del orquestador persiste todas las decisiones en una única transacción al finalizar el flujo.

Las funciones principales del repositorio son:

- `save_claim(...)`: inserta un expediente si no existe o actualiza su estado. Operación idempotente y segura ante reintentos.
- `log_agent_decision(claim_id, agent, action, reasoning, confidence, hitl_required)`: materializa en base de datos la decisión de cada agente, incluyendo el texto de razonamiento generado por `reason()`.
- `get_claim_with_decisions(claim_id)`: devuelve el expediente completo con todas sus decisiones en orden cronológico, para su consulta a través de la API REST.
- `list_claims(status, limit, offset)`: lista expedientes con paginación y filtro opcional por estado.

Esta separación entre herramientas (acciones sobre el mundo externo) y persistencia (efecto secundario interno gestionado por el repositorio) sigue el principio de separación de responsabilidades y facilita la sustitución de la base de datos subyacente sin modificar la lógica de los agentes.

3.7 Tabla consolidada: visión a producción

La tabla siguiente resume el camino de integración proyectado para cada herramienta y capacidad del sistema, ofreciendo una visión de conjunto de los sistemas reales que deberían emplearse en una versión productiva de Smart-Claims Agent.

Herramienta / Capacidad	Estado en el prototipo	Sistema / Tecnología en producción
<code>validate_documents</code>	Mock externo	ECM de Seguros Pepín + API del core asegurador (verificación de póliza activa)
<code>extract_multimodal</code>	Mock externo (fallback del Agente C)	Sustituido por Claude Vision en el camino principal; fallback a Tesseract OCR
Claude Vision (<code>analyze_document</code>)	IA real (gated por <code>ANTHROPIC_API_KEY</code>)	Sin cambios; escalado mediante gestión de concurrencia y cuota de la API
<code>check_policy</code>	Mock externo (fallback del Agente D)	Sustituido por RAG en el camino principal
RAG de pólizas (<code>retrieve_policy</code>)	IA real (gated por <code>SCA_RAG_ENABLED</code>)	Corpus de pólizas reales de Seguros Pepín + ChromaDB persistido (servidor dedicado)
Motor antifraude (4 detectores)	IA real (determinista)	Baselines calculados desde BD histórica; OFAC SDN oficial; duplicados contra MariaDB
<code>approve_payment</code>	Mock externo	Pasarela de pagos / módulo financiero del core asegurador de Seguros Pepín
<code>send_rejection</code>	Mock externo	CRM / sistema de notificaciones corporativo de Seguros Pepín
<code>request_more_info</code>	Mock externo	Portal del cliente + proveedor de correo electrónico transaccional

Bibliografía

Anthropic. (2024). *Tool use (function calling) — Claude API documentation*. <https://docs.anthropic.com/en/docs/tool-use>

Chase, H. (2022). *LangChain* [Software]. <https://github.com/langchain-ai/langchain>

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Oguz, B., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing reasoning and acting in language models*. International Conference on Learning Representations (ICLR 2023). <https://arxiv.org/abs/2210.03629>

ewpage

Capítulo 3 — Manual de usuario

Manual de usuario — Smart-Claims Agent

Seguros Pepín, S.A. — MVP agéntico de gestión de siniestros

TFM Máster en Machine Learning e Inteligencia Artificial, OBS Business School

Índice

1. [Introducción y público objetivo](#)
 2. [Requisitos previos](#)
 3. [Configuración del entorno](#)
 4. [Modo 1 — Docker Compose \(backend, base de datos y RAG\)](#)
 5. [Modo 2 — App Streamlit \(demo principal\)](#)
 6. [Modo 3 — CLI de demostración, API REST y evaluador](#)
 7. [Interpretación de resultados](#)
 8. [Inspección de la base de datos con Adminer](#)
 9. [Resolución de problemas frecuentes](#)
 10. [Referencias](#)
-

1. Introducción y público objetivo

Este manual describe el modo de operación del prototipo **Smart-Claims Agent** en su versión de entrega del TFM (estado del repositorio a 18 de agosto de 2026). No es un manual orientado al empleado final de Seguros Pepín, S.A., sino un **manual operativo para el evaluador técnico** (director del TFM, tribunal académico o desarrollador que revise el prototipo). Su objetivo es permitir reproducir, inspeccionar y validar el comportamiento del sistema de forma autónoma.

El sistema puede operarse de tres formas complementarias:

- **Modo 1 — Docker Compose:** levanta el backend FastAPI junto con la infraestructura de datos (MariaDB, ChromaDB y Adminer) en contenedores aislados, con persistencia real en base de datos. Es la vía para usar la API REST y auditar las decisiones en SQL.

- **Modo 2 — App Streamlit (la demo principal):** interfaz web autónoma que invoca el grafo de agentes directamente en el mismo proceso Python, sin necesidad de Docker ni de MariaDB. Es la modalidad desplegada en Streamlit Community Cloud y la recomendada para la demostración en vivo ante el tribunal. Incluye siete vistas: Inicio, Bandeja, Nueva reclamación, Conciliación (Agente F), Historial, Arquitectura y Caso libre.
- **Modo 3 — CLI de demostración, API REST y evaluador:** la CLI ejecuta cuatro casos predefinidos desde línea de comandos y muestra el Chain of Thought; la API REST acepta peticiones HTTP desde curl, Postman o cualquier cliente; el evaluador in-process reproduce las métricas del capítulo 4 sobre el dataset sintético.

Nota sobre integraciones externas: todas las herramientas que en producción consultarían sistemas reales de Seguros Pepín (gestor documental, núcleo de pólizas, sistemas de pago, listas oficiales de sanciones) están implementadas como **mocks deterministas**, ya que el proyecto académico no tiene acceso a esos sistemas. En cambio, las **capacidades de IA del proyecto sí son reales**: la extracción multimodal (Agente C, Claude Vision), el RAG de cobertura (Agente D, ChromaDB) y el motor antifraude (Agente G) operan de verdad, **sobre datos sintéticos** (pólizas, lista OFAC y baselines de prototipo). El LLM Claude de Anthropic es **opcional**: si se proporciona `ANTHROPIC_API_KEY`, cada agente genera razonamientos Chain of Thought con `claude-sonnet-4-6` y el Agente C realiza extracción multimodal real; sin clave, el sistema usa un *fallback* determinista y la demo decide de forma idéntica.

Sobre las capturas de pantalla: todas las figuras de este capítulo se han generado sobre la app Streamlit ejecutada en local (Streamlit 1.57.0, Python 3.11) con la clave de Anthropic configurada, de modo que muestran el modo «Claude activo». Sin clave, la disposición de las pantallas es idéntica; solo cambian el texto del razonamiento y la sección de extracción multimodal (véase §3.3).

2. Requisitos previos

2.1 Modo 1 — Docker Compose

Requisito	Versión mínima	Notas
Docker Engine	24.x	Incluye el demonio de contenedores
Docker Compose	v2 (plugin integrado)	Comando <code>docker compose</code> sin guion
RAM disponible	4 GB	Para los cuatro servicios en paralelo
Puertos libres	8000, 8080, 8082, 3306	Véase tabla de servicios en §4.2

Verificación rápida:

```
docker --version
docker compose version
```

2.2 Modo 2 — App Streamlit (local) y Modo 3 — CLI

Requisito	Versión	Notas
Python	3.11	Versión fijada en <code>.python-version (3.11.16)</code> , que es la que usa Streamlit Community Cloud; 3.12 también es compatible

Requisito	Versión	Notas
Dependencias (raíz)	—	<code>requirements.txt</code> de la raíz del repositorio (Streamlit 1.57.0, LangGraph 1.2.1, langchain-anthropic 1.4.3, chromadb 1.5.9, pandas 2.2.3)
Dependencias (backend)	—	<code>backend/requirements.txt</code> (para la CLI, los tests y la API sin Docker)
Navegador	Reciente	Chrome, Edge o Firefox actualizados
Conexión a internet	Opcional	Solo si se configura <code>ANTHROPIC_API_KEY</code> (llamadas a la API de Anthropic)

Instalación de dependencias (una sola vez):

```
# Windows – desde la raíz del repositorio
py -m pip install -r requirements.txt
```

```
# Linux / macOS
python3.11 -m pip install -r requirements.txt
```

3. Configuración del entorno

3.1 Crear el fichero `.env`

El fichero `.env` es la fuente de configuración del sistema. Se parte de la plantilla incluida en el repositorio:

```
# Bash (Linux / macOS)
cp .env.example .env
```

```
# PowerShell (Windows)
Copy-Item .env.example .env
```

3.2 Variables de entorno relevantes

Editar `.env` con los valores adecuados. La tabla siguiente describe las variables más importantes tal como aparecen en `.env.example`:

Variable	Valor por defecto en <code>.env.example</code>	Descripción
<code>ANTHROPIC_API_KEY</code>	<code>sk-ant-api03-XXXX...</code> (placeholder)	Clave API de Anthropic. Opcional — véase §3.3
<code>HITL_AMOUNT_THRESHOLD</code>	<code>5000.0</code>	Umbral (€) por encima del cual se activa la revisión humana (HITL)

Variable	Valor por defecto en .env.example	Descripción
SCA_RAG_ENABLED	1	1 = el Agente D usa RAG real sobre ChromaDB; vacío o 0 = catálogo determinista
DB_USER	claims_user	Usuario de aplicación de MariaDB
DB_PASSWORD	claims_s3cret_dev	Contraseña del usuario de aplicación
DB_HOST	mariadb	Hostname del servicio MariaDB (nombre del contenedor en Docker)
DB_PORT	3306	Puerto de MariaDB
DB_NAME	smart_claims	Nombre de la base de datos
DB_ROOT_PASSWORD	root_s3cret_dev	Contraseña root de MariaDB
CHROMA_HOST	chromadb	Hostname del servicio ChromaDB
CHROMA_PORT	8000	Puerto interno de ChromaDB
CHROMA_COLLECTION	pepin_policies	Colección vectorial con las pólizas de Seguros Pepín
BACKEND_URL	http://backend:8000	URL interna del backend (entre contenedores Docker)
ENVIRONMENT	development	Entorno de ejecución
LOG_LEVEL	INFO	Nivel de log del backend

Nota sobre moneda: los importes se expresan en euros (€) como simplificación del prototipo; en una implantación para Seguros Pepín se localizarían a pesos dominicanos (DOP / RD\$). La app Streamlit muestra los importes con separador de millares anglosajón (5,000 €), por el formato numérico por defecto de Python.

3.3 La clave `ANTHROPIC_API_KEY` y el modo fallback

Esta variable controla el nivel de inteligencia real del sistema:

- **Con clave válida:** cada agente llama a `claude-sonnet-4-6` (temperatura 0, tiempo máximo de 20 s por llamada) para generar el razonamiento Chain of Thought. El Agente C realiza extracción multimodal real (tipo, importe, fecha, proveedor, resumen y confianza) sobre los documentos subidos mediante Claude Vision. Requiere conexión a internet y saldo en la cuenta de Anthropic. Un expediente completo tarda entre 20 y 60 segundos.
- **Sin clave (o clave vacía):** el módulo de razonamiento detecta la ausencia de la variable y retorna texto de fallback predefinido. **La decisión final del orquestador es idéntica en ambos modos**, ya que la lógica de enrutamiento es determinista (basada en validación de entrada, documentos aportados, fraude, cobertura e importe). Un expediente se resuelve en menos de un segundo. Para la evaluación académica del prototipo, el modo fallback es suficiente.

La app Streamlit muestra en la parte inferior de la barra lateral el indicador de modo activo:

-  `Claude activo (CoT enriquecido)` — con clave.
-  `Modo fallback determinista (sin clave)` — sin clave.

En Streamlit Community Cloud, la clave se inyecta vía la sección *Secrets* del panel de administración de la app (véase [docs/DEPLOY-STREAMLIT.md](#)), nunca como variable de entorno del repositorio. El fichero `.streamlit/secrets.toml` está excluido del control de versiones por `.gitignore`.

3.4 Variable `SCA_RAG_ENABLED` y el Agente D

Cuando `SCA_RAG_ENABLED=1`, el Agente D (Verificación de cobertura) consulta una colección ChromaDB **embebida** (sin servidor: se indexan al arrancar los cuatro documentos de póliza sintéticos de `data/policies/SP-PCS-009-*.md`) para recuperar el fragmento de póliza más relevante según el tipo de siniestro. Si ChromaDB no está disponible o la variable está vacía/a `0`, el agente cae automáticamente al catálogo determinista sin interrumpir el flujo.

La app Streamlit activa `SCA_RAG_ENABLED=1` por defecto mediante `os.environ.setdefault("SCA_RAG_ENABLED", "1")` al arrancar, por lo que la demo funciona con RAG real sin ninguna configuración adicional.

4. Modo 1 — Docker Compose (backend, base de datos y RAG)

4.1 Arranque del sistema

Desde la **raíz del repositorio**, con el fichero `.env` configurado:

```
docker compose up -d --build
```

Docker Compose construye la imagen local del backend, descarga el resto de imágenes (`chromadb/chroma:0.5.3`, `mariadb:11.3`, `adminer:4.8.1`) y levanta los servicios en segundo plano. El primer arranque puede tardar entre 2 y 5 minutos.

Para seguir los logs del backend en tiempo real:

```
docker compose logs -f backend
```

4.2 Servicios y URLs

Servicio	Contenedor	Puerto host	URL	Descripción
Backend FastAPI	<code>sca-backend</code>	8000	<code>http://localhost:8000</code>	API REST + orquestador LangGraph
ChromaDB	<code>sca-chromadb</code>	8080	<code>http://localhost:8080</code>	Vector store para RAG de pólizas
MariaDB	<code>sca-mariadb</code>	3306	<code>localhost:3306</code>	Persistencia relacional
Adminer	<code>sca-adminer</code>	8082	<code>http://localhost:8082</code>	Inspector web de la BD

El servicio `frontend` está desactivado. Desde el 18 de agosto de 2026 el bloque `frontend` de `docker-compose.yml` está comentado: el antiguo dashboard `frontend/app.py` (que consumía la API por HTTP) ha sido sustituido por la app autónoma `streamlit_app.py`, que se ejecuta por separado (\$5) y no necesita Docker. Por tanto, `docker compose up` **no** publica nada en el puerto 8501.

El contenedor `sca-backend` espera a que MariaDB supere su healthcheck antes de iniciarse (condición `service_healthy` en `docker-compose.yml`). Si el backend aparece como `restarting` en los primeros 30-60 segundos, es comportamiento normal.

4.3 Verificación del sistema

Verificar que el backend responde:

```
curl http://localhost:8000/health
```

Respuesta esperada:

```
{"status": "ok", "version": "0.5.0"}
```

Consultar el estado de los seis agentes:

```
curl http://localhost:8000/api/v1/agents/status
```

Respuesta esperada (resumen):

```
{
  "pattern": "Supervisor (Hub-and-Spoke) sobre LangGraph",
  "agent_count": 6,
  "agents": [
    {"id": "A", "name": "Orchestrator", "status": "operational"},
    {"id": "B", "name": "Document Validator", "status": "operational"},
    {"id": "C", "name": "Multimodal Extractor", "status": "operational"},
    {"id": "D", "name": "Coverage Checker", "status": "operational"},
    {"id": "E", "name": "Claim Resolver", "status": "operational"},
    {"id": "G", "name": "Fraud Compliance", "status": "operational"}
  ]
}
```

La documentación Swagger interactiva está disponible en <http://localhost:8000/docs>.

4.4 Parada del sistema

```
# Detener los contenedores (conserva los volúmenes de datos)
docker compose down

# Detener y eliminar también los volúmenes
docker compose down -v
```

5. Modo 2 — App Streamlit (demo principal)

La app `streamlit_app.py` es la interfaz de demostración del prototipo. Ejecuta el grafo de agentes **en el mismo proceso** (importa `process_claim` del orquestador), de modo que no necesita backend, MariaDB ni Docker. La persistencia en base de datos es *best-effort*: si no hay MariaDB, el orquestador registra un aviso en el log y el resultado se muestra igualmente.

5.1 Arranque

En local, desde la raíz del repositorio:

```
# Windows
py -m streamlit run streamlit_app.py
```

```
# Linux / macOS
python3.11 -m streamlit run streamlit_app.py
```

Streamlit abre el navegador automáticamente en `http://localhost:8501`. Si el puerto está ocupado, añadir `--server.port 8600` (o cualquier otro).

En Streamlit Community Cloud, la app está desplegada desde el repositorio `FreeBarnOwl/smart-claims-agent-vfull` (rama `main`, fichero principal `streamlit_app.py`), con `ANTHROPIC_API_KEY` y `HITL_AMOUNT_THRESHOLD` definidos en *Secrets*. El procedimiento de despliegue y actualización está descrito en `docs/DEPLOY-STREAMLIT.md`. Las apps gratuitas de Streamlit Cloud se «duermen» tras varios días sin uso: la primera visita puede tardar 1-2 minutos en despertar la app.

El tema visual (fondo claro, azul corporativo) está fijado en `.streamlit/config.toml`, por lo que la app se ve igual con independencia del tema del navegador.

5.2 Estructura de la interfaz

NAVEGACIÓN

Inicio

Bandeja

Nueva reclamación

Conciliación (Agente F)

Historial

Arquitectura

Caso libre

Claude activo (CoT enriquecido)

SEGUROS PEPÍN

Smart-Claims Agent

Gestión agéntica de siniestros
Seguros Pepín, S.A.

Centro de Gestión de Siniestros

Bienvenido. Selecciona la acción que quieres realizar.



Nueva reclamación

Registra un expediente y procésalo con los seis agentes. Verás la decisión final y el razonamiento (Chain of Thought).

Empezar



Historial

Consulta los expedientes procesados en esta sesión, con su estado, decisión y distribución.

Ver historial



Escenarios de demostración

Lanza con un clic los cinco casos representativos del flujo (pago, revisión humana, información, rechazo y bloqueo por fraude).

Ir a escenarios

Figura 3.1. Vista de Inicio. A la izquierda, la barra lateral de navegación con las siete vistas y el indicador de modo (Claude activo / fallback); a la derecha, la cabecera corporativa y las tarjetas de acceso rápido.

La interfaz se compone de dos zonas fijas:

- **Barra lateral (izquierda):** sección *Navegación* con siete botones — Inicio, Bandeja, Nueva reclamación, Conciliación (Agente F), Historial, Arquitectura, Caso libre — y, bajo el separador, el indicador de modo LLM (§3.3). El botón de la vista activa aparece resaltado.
- **Área principal:** cabecera «Seguros Pepín · Smart-Claims Agent» y el contenido de la vista seleccionada.

Vista	Para qué sirve	Sección
Inicio	Menú de bienvenida con accesos rápidos a Nueva reclamación, Historial y Escenarios de demostración	§5.3
Bandeja	Simula la bandeja de entrada de siniestros recibidos por WhatsApp, con adjuntos reales (fotos y factura PDF) que el Agente C lee con Claude Vision	§5.7
Nueva reclamación	Cinco escenarios de un clic (uno por camino del flujo) y formulario personalizado con subida de documentos	§5.4
Conciliación (Agente F)	Demostrador secundario del asesor de conciliación sobre tres expedientes en negociación	§5.8
Historial	Tabla y gráfico de los expedientes procesados en la sesión	§5.9
Arquitectura	Descripción de los seis agentes, del módulo aparte (Agente F) y de las características clave	§5.10

Vista	Para qué sirve	Sección
Caso libre	Panel de reserva con campos de texto libre para probar el blindaje de entrada con datos improvisados	§5.11

El resultado no persiste al cambiar de vista. Al pulsar cualquier botón de navegación se descarta el último resultado en pantalla (`last_result`); el expediente sigue disponible en *Historial*. Es un comportamiento deliberado para que ningún resultado antiguo se muestre por error en otra vista.

5.3 Vista Inicio

Tres tarjetas con botón: **Nueva reclamación** (*Empezar*), **Historial** (*Ver historial*) y **Escenarios de demostración** (*Ir a escenarios*, que lleva a la misma vista de Nueva reclamación). La tarjeta de *Arquitectura* está temporalmente oculta en Inicio; la vista sigue accesible desde la barra lateral.

5.4 Vista Nueva reclamación

NAVEGACIÓN

Inicio

Bandeja

Nueva reclamación

Conciliación (Agente F)

Historial

Arquitectura

Caso libre

SEGUROS PEPÍN

Smart-Claims Agent

Gestión agéntica de siniestros Seguros Pepín, S.A.

Nueva reclamación

ESCENARIOS RÁPIDOS

Pago automático

Daños propios · 2.500 € · documentación completa → PAGO

Procesar

Revisión humana (HITL)

Responsabilidad civil · 9.500 € → REVISIÓN HUMANA por importe

Procesar

Información requerida

Daños propios · faltan documentos → SOLICITUD DE INFO

Procesar

Rechazo por no cobertura

Daños mecánicos · sin cobertura en póliza → RECHAZO

Procesar

Bloqueo por fraude (OFAC)

Asegurado en la lista de sanciones (OFAC/SDN) · documentación completa → RECHAZO_FRAUDE

Procesar

O CREA UNA RECLAMACIÓN PERSONALIZADA

Nombre del asegurado

p. ej. Juan García

ID Cliente

p. ej. CLIENT-A

Email del cliente

cliente@ejemplo.com

Tipo de siniestro

Selecciona el tipo...

Importe reclamado (€)

0

Documentos aportados (tipo)

Choose options

Sube los documentos reales (factura, foto de daños, acta...) — el Agente C los analizará con Claude Vision

Upload

200MB per file · PNG, JPG, WEBP, PDF

Procesar reclamación

Figura 3.2. Vista Nueva reclamación: cinco escenarios rápidos (arriba) y formulario personalizado, vacío por defecto (abajo).

33 / 68

5.4.1 Escenarios rápidos

Cinco tarjetas, una por cada camino posible del flujo. Pulsar **Procesar** bajo la tarjeta lanza el expediente con los datos indicados (cliente **CLIENT-DEMO**) y muestra el resultado bajo el formulario.

Escenario	claim_type	Importe	Documentos	Nombre del asegurado	Decisión esperada
Pago automático	danys_propis	2.500 €	foto_danys, factura, denuncia_companyia	—	PAGO (2.200 € pagados: 2.500 – 300 de franquicia)
Revisión humana (HITL)	responsabilitat	9.500 €	foto_danys, acta_policial, dades_tercer	—	REVISION_HUMANA (9.500 € > umbral 5.000 €)
Información requerida	danys_propis	3.000 €	Solo factura	—	INFO_REQUERIDA (faltan foto_danys, denuncia_companyia)
Rechazo por no cobertura	danys_mecanics	1.500 €	informe_taller, factura	—	RECHAZO (exclusión SP-PCS-009 § 7.3)
Bloqueo por fraude (OFAC)	danys_propis	2.500 €	Completos	Viktor Nikolaev Kozlov	RECHAZO_FRAUDE (coincidencia con la lista de sanciones sintética)

5.4.2 Formulario personalizado

Todos los campos arrancan **vacíos** (solo *placeholders*), de manera que el evaluador introduce exactamente lo que quiere probar:

Campo	Tipo de control	Comportamiento
Nombre del asegurado	Texto (p. ej. Juan García)	Opcional. Se contrasta con la lista de sanciones (Agente G)
ID Cliente	Texto (p. ej. CLIENT-A)	Opcional; por defecto CLIENT-A . Los IDs C-A , C-B y C-C tienen historial simulado (§7.3)
Email del cliente	Texto	Opcional; por defecto cliente@segurospepin.com
Tipo de siniestro	Desplegable	Obligatorio. Cuatro tipos válidos (§5.4.3)
Importe reclamado (€)	Numérico, 0–100.000, paso 100	Obligatorio (> 0). Para importes fuera de ese rango, usar <i>Caso libre</i> (§5.11)
Documentos aportados (tipo)	Selección múltiple	Tipos declarados del expediente; determinan INFO_REQUERIDA
Sube los documentos reales	Subida de ficheros (PNG, JPG, WEBP, PDF; hasta 200 MB/fichero)	Opcional. Si hay clave, el Agente C los analiza con Claude Vision

Si se pulsa **Procesar reclamación** sin tipo o sin importe, la app muestra el aviso «Indica al menos el tipo de siniestro y el importe reclamado.» y no lanza el flujo.

Los ficheros subidos se leen en memoria y se envían a Claude Vision en la misma petición; no se guardan en disco. Para una prueba rápida con documentos reales sin subir nada, la vista *Bandeja* (§5.7) ya incluye adjuntos.

5.4.3 Tipos de siniestro y documentación requerida

El Agente B (validación documental) exige, por tipo, los siguientes documentos declarados. Cualquier ausencia produce **INFO_REQUERIDA** con la lista de los que faltan.

claim_type	Etiqueta en la app	Documentos requeridos	Cobertura (Agente D, catálogo SP-PCS-009)
danys_propis	Daños propios	foto_danys, factura, denuncia_companyia	Cubierto · máx. 10.000 € · franquicia 300 € · § 3.2
responsabilitat	Responsabilidad civil	foto_danys, acta_policial, dades_tercer	Cubierto · máx. 50.000 € · sin franquicia · § 4.1
robatori	Robo	acta_policial, llista_objectes_robats	Cubierto · máx. 8.000 € · franquicia 500 € · § 5.0
danys_mecanics	Daños mecánicos	informe_taller, factura	No cubierto · § 7.3 (exclusión)

5.5 Lectura del resultado



Figura 3.3. Resultado del escenario «Pago automático»: cabecera con el número de expediente y la píldora de decisión, motivo de terminación, recorrido por los agentes (stepper), tarjetas de métricas, cribado antifraude, cobertura RAG y cadena de razonamiento.

Todas las vistas que procesan expedientes (Nueva reclamación, Bandeja y Caso libre) muestran el resultado con la misma estructura, de arriba abajo:

- 1. **Cabecera Expediente CLM-XXXXXXX** — identificador generado (8 caracteres hexadecimales) — y la **píldora de decisión**, con cinco posibles textos:

Píldora	Color	Código interno
Resuelto · Pago aprobado	Verde	PAGO
Revisión humana requerida	Ámbar	REVISION_HUMANA
Información requerida	Azul	INFO_REQUERIDA
Rechazado · Sin cobertura	Rojo	RECHAZO
Bloqueado · Fraude / OFAC	Rojo	RECHAZO_FRAUDE

2. **Motivo de terminación** (texto pequeño bajo la cabecera): la razón literal con la que el orquestador cerró el expediente, por ejemplo *pago aprobado, importe 9500.0 EUR supera umbral HITL (5000.0 EUR), documentacion incompleta: faltan foto_danys, denuncia_companyia, rechazado por no cobertura o caso bloqueado por fraude/OFAC*.
3. **Recorrido por los agentes (stepper A → B → C → G → D → E)**: cada agente que intervino aparece en verde, el último que actuó en azul y los no ejecutados en gris. Permite ver de un vistazo en qué punto terminó el flujo (§5.6).
4. **Cuatro tarjetas de métricas**: *Estado (resolved, pending_review, validating, rejected)*, *Decisión* (código interno), *Importe pagado* (con el subtítulo «de N € solicitados») y *Tiempo* de procesamiento en segundos.
5. **Cribado antifraude (Agente G)**: píldora con el veredicto (*CLEAR, MEDIUM_RISK, HIGH_RISK, BLOCKED*) y la puntuación de riesgo 0–1, más las señales detectadas (§7.3). Solo aparece si el flujo llegó al Agente G.
6. **Cobertura (Agente D · RAG sobre pólizas)**: sección y fragmento de la póliza recuperados de ChromaDB, con la distancia de recuperación. Solo aparece cuando la fuente es *rag* (si el Agente D usó el catálogo determinista, se omite).
7. **Extracción multimodal real (Agente C · Claude Vision)**: una tarjeta por documento subido con *Importe leído, Fecha, Confianza (%)* y resumen. Solo aparece si se subieron ficheros y hay clave de Anthropic (§7.5). Debajo se recuerda que el importe de la decisión es el declarado en el formulario, no el leído en el documento.
8. **Cadena de razonamiento de los agentes**: línea de tiempo con una tarjeta por agente (*decisions_log*), con su acción y el razonamiento Chain of Thought (generado por Claude o texto de fallback).

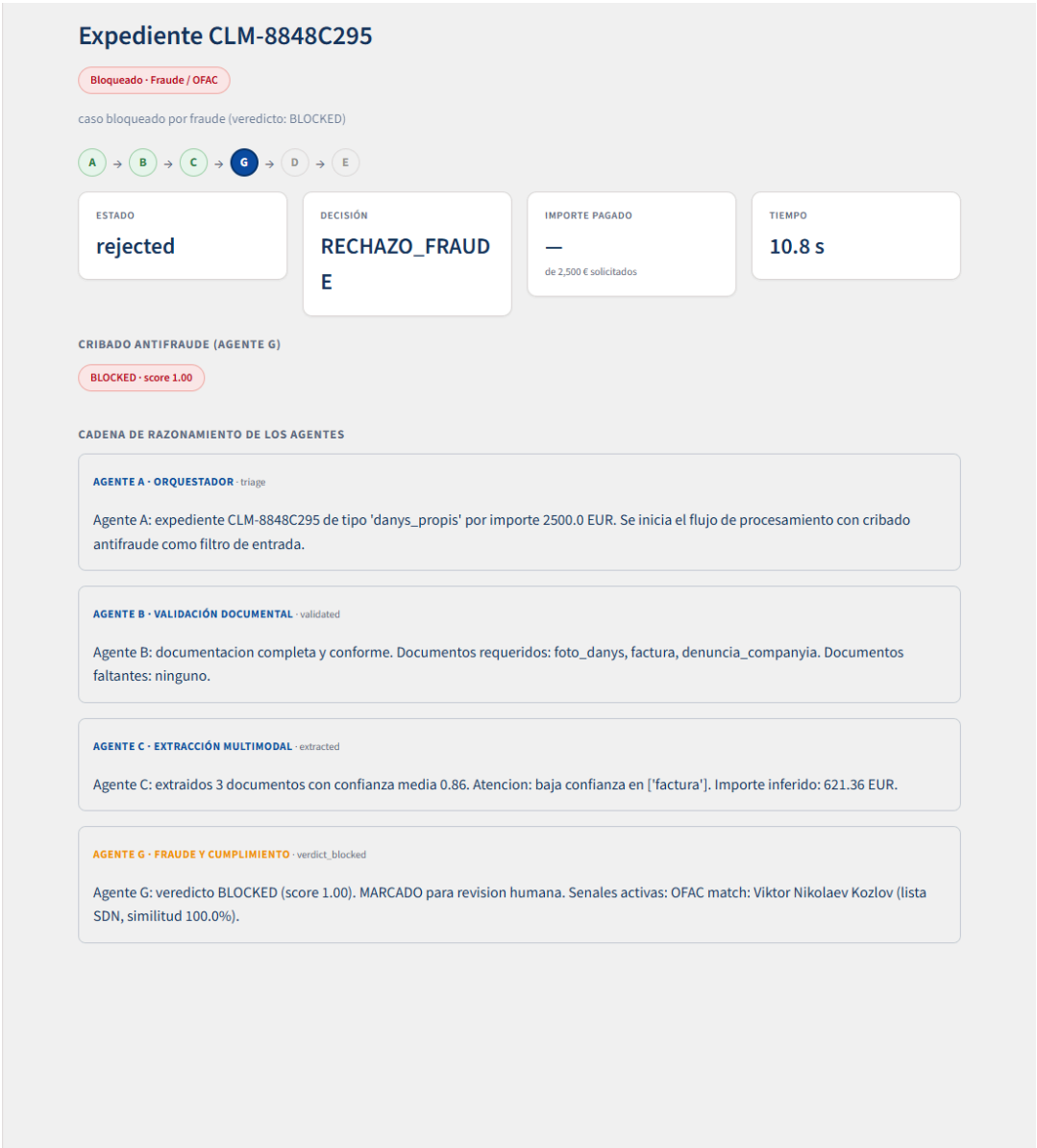


Decisión	Agentes ejecutados	Dónde termina	Cómo dispararlo
RECHAZO_FRAUDE	A, B, C, G	El Agente G devuelve HIGH_RISK o BLOCKED	Escenario «Bloqueo por fraude (OFAC)» o un nombre de asegurado parecido a una entrada de la lista de sanciones sintética (§7.3). Un importe anómalo por sí solo suma 0,40 y produce MEDIUM_RISK , que no bloquea
RECHAZO	A, B, C, G, D	El Agente D no encuentra cobertura	Escenario «Rechazo por no cobertura» o cualquier danys_mechanics
REVISION_HUMANA	A, B, C, G, D, E	El Agente E calcula un importe neto superior a 5.000 €	Escenario «Revisión humana», o danys_propis con importe > 5.300 €, o responsabilitat > 5.000 €
PAGO	A, B, C, G, D, E	El Agente E aprueba el pago (importe neto ≤ 5.000 €)	Escenario «Pago automático» o cualquier caso cubierto, documentado, sin fraude y por debajo del umbral









<<

NAVEGACIÓN

Inicio

Bandeja

Nueva reclamación

Conciliación (Agente F)

Historial

Arquitectura

Caso libre

Claude activo (CoT enriquecido)

SEGUROS PEPÍN

Smart-Claims Agent

Gestión agéntica de siniestros
Seguros Pepín, S.A.

Bandeja de entrada

Casos recibidos por canales conectados a la aseguradora (aquí, WhatsApp mediante un conector simulado). El gestor comprueba que la documentación esté completa antes de disparar el procesado.

CLM-WA-0001

Marta Soler Puig

Daños propios

3,200 €

WhatsApp - conector simulado

Entrada: 08/08/2026 09:14



Hola buenas! Ayer choqué contra un poste al aparcar y se llevó el parachoques y el faro delantero 🚗 Ayer mismo abrí el reclamo por teléfono y me dieron el número de denuncia D-4521. Os mando las fotos del golpe y la factura del taller, gracias!

Adjuntos recibidos



Foto daño frontal



Foto daño lateral



Factura del taller

factura_taller_can_bosch.pdf

Descargar PDF

Documentación completa para este tipo de siniestro.

Revisar y procesar

CLM-WA-0002

Jordi Ferrer Camps

Daños propios

2,900 €

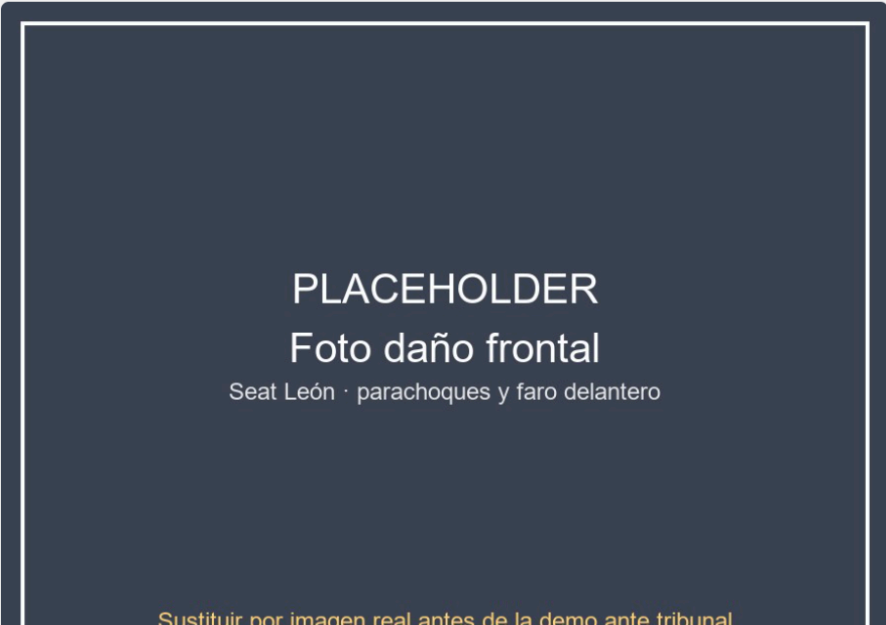
WhatsApp - conector simulado

Entrada: 08/08/2026 11:02



Buenos días, tuve un golpe con el coche este finde y quiero reclamarlo. Os paso una foto de cómo quedó, luego os mando la factura del taller y el número de denuncia en cuanto los tenga.

Adjuntos recibidos



Sustituir por imagen real antes de la demo ante tribunal

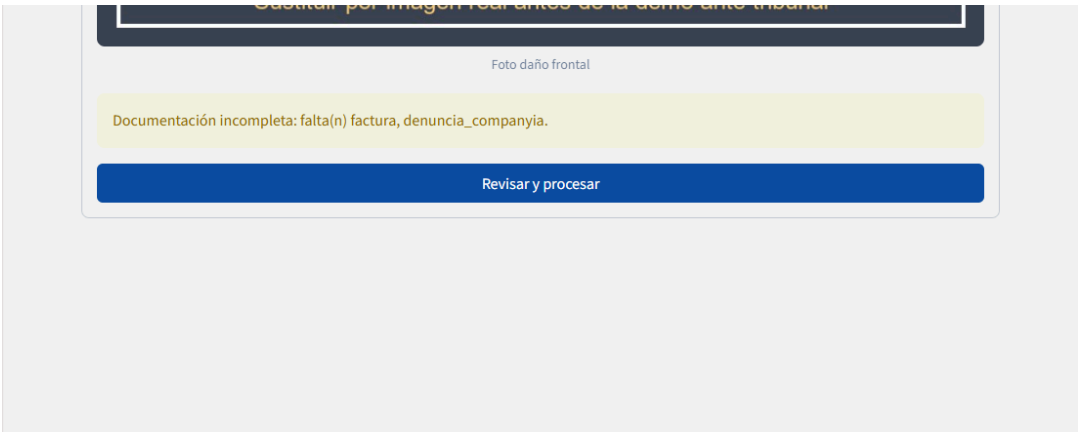


Figura 3.8. Bandeja de entrada simulada: dos siniestros recibidos por WhatsApp con su mensaje, adjuntos (fotos y factura PDF con miniatura) y el semáforo de documentación.

La Bandeja simula la recepción de siniestros por un canal conversacional (WhatsApp). Cada tarjeta muestra el identificador del ticket, el asegurado, la fecha de entrada, el tipo, el importe, el mensaje literal del cliente, los **adjuntos reales** (imágenes en `assets/demo/`, y la factura PDF con miniatura y botón `Descargar PDF`) y un aviso de completitud documental:

Ticket	Asegurado	Tipo / importe	Adjuntos	Documentación	Resultado esperado
CLM-WA-0001	Marta Soler Puig (CLIENT-BANDEJA-01)	Daños propios · 3.200 €	Foto frontal, foto lateral, <code>factura_taller_can_bosch.pdf</code>	Completa (incluye denuncia D-4521)	PAGO · 2.900 € (3.200 – 300)
CLM-WA-0002	Jordi Ferrer Camps	Daños propios · 2.900 €	Solo foto de daños	Incompleta: falta(n) <code>factura</code> , <code>denuncia_companyia</code>	INFO_REQUERIDA

El botón `Revisar y procesar` lanza el expediente **con los adjuntos como ficheros subidos**, por lo que, con clave de Anthropic, el Agente C realiza extracción real sobre las dos fotos y el PDF. El resultado se muestra **dentro de la propia tarjeta** del ticket procesado, para que la narración de la demo quede junto al mensaje que la originó.

Claude activo (CoT enriquecido)

Expediente CLM-C392CAE6

Resuelto · Pago aprobado

pago aprobado

A → B → C → G → D → E

ESTADO

resolved

DECISIÓN

PAGO

IMPORTE PAGADO

2,900 €

de 3,200 € solicitados

TIEMPO

114.2 s

CRIBADO ANTIFRAUDE (AGENTE G)

CLEAR

COBERTURA (AGENTE D · RAG SOBRE PÓLIZAS)

Recuperado por RAG · SP-PCS-009 § 3.2

Siniestro tipo danyys_propis. danyys_propis. Daños propios del vehículo asegurado por colisión, accidente, vuelco o impacto. Cubierto, límite 10000 EUR, franquicia 300 EUR.

EXTRACCIÓN MULTIMODAL REAL (AGENTE C · CLAUDE VISION)

foto_danyys_frontal_placeholder.png · foto_danos

IMPORTE LEÍDO

—

FECHA

—

CONFIANZA

40%

Imagen placeholder que sustituye a una fotografía real de daños frontales en un Seat León, concretamente en el parachoques y faro delantero. La imagen real debe incorporarse antes de la demostración ante tribunal.

foto_danyys_lateral_placeholder.png · foto_danos

IMPORTE LEÍDO

—

FECHA

—

CONFIANZA

40%

Placeholder de fotografía de daño lateral de un Seat León, específicamente en la chapa lateral izquierda. La imagen real debe ser sustituida antes de la demo ante tribunal.

factura_taller_can_bosch.pdf · factura

IMPORTE LEÍDO

3,200 €

FECHA

2026-08-06

CONFIANZA

98%

Factura de reparación de vehículo Seat León (matrícula 4521-BXX) perteneciente a Marta Soler Puig, por daños propios (ref. D-4521). Incluye parachoques delantero con pintura (795€), faro delantero derecho (310€), reparación y pintura de chapa lateral izquierda (1.150€) y 9 horas de mano de obra (390€). Base imponible 2.645€, IVA 21% de 555,45€, total 3.200,45€.

El importe de la decisión es el indicado en el formulario; arriba se muestra lo que Claude ha leído de los documentos subidos.

CADENA DE RAZONAMIENTO DE LOS AGENTES

AGENTE A · ORQUESTADOR · triage

Análisis de Triage — Expediente CLM-C392CAE6

Paso 1: Identificación del Tipo de Siniestro

Tipo declarado: danyys_propis (daños propios) → Se trata de una reclamación por daños sufridos en el propio vehículo/propiedad del asegurado, no daños a terceros. → Cobertura habitual: seguro a todo riesgo o cobertura específica de daños propios.

Paso 2: Evaluación del Importe

Importe reclamado: 3.200,00 € → Importe moderado-medio. No supera umbrales de alerta alta (habitualmente >10.000 €), pero

Figura 3.9. Resultado de *CLM-WA-0001* procesado desde la Bandeja (*PAGO* · 2.900 € de 3.200 €): además de las secciones habituales, aparece «Extracción multimodal real (Agente C · Claude Vision)» con una tarjeta por adjunto. Claude lee de la factura del taller el importe (3.200 €), la fecha (2026-08-06) y una confianza del 98 %, y describe su contenido (matrícula, conceptos, base imponible e IVA); en las dos fotografías, que en los fixtures son imágenes placeholder, no

46 / 68

encuentra importe ni fecha y asigna una confianza del 40 %, por debajo del umbral de 0,85 (\$7.5), lo que las marca como de baja confianza. La decisión se toma sobre el importe declarado en el ticket, como indica la nota al pie de la sección.

5.8 Vista Conciliación (Agente F)

NAVEGACIÓN

Inicio

Bandeja

Nueva reclamación

Conciliación (Agente F)

Historial

Arquitectura

Caso libre

Claude activo (CoT enriquecido)

SEGUROS PEPÍN

Smart-Claims Agent

Gestión agéntica de siniestros Seguros Pepín, S.A.

Conciliación (Agente F)

Demostrador secundario, independiente del flujo principal A→B→C→G→D→E: el caso base de este prototipo (daños propios) no tiene fase de negociación. Esta vista aplica el Agente F sobre tres expedientes DPA/RC (daños propios / responsabilidad civil) ya en proceso de conciliación con el cliente.

El Agente F solo recomienda por reglas explicables (sin aprendizaje automático): no ejecuta ninguna acción de negocio, no cambia estados ni persiste decisiones. Cada recomendación la ejecuta un gestor humano.

CLM-CONC-0001

DPA

8,000 € reclamados

Oferta pendiente de subir de tramo

Oferta al 65% ya hecha

Prioridad baja

Recomendación: Subir al siguiente tramo (75% = 6,000.00 EUR). El Agente F solo recomienda: la ejecuta un gestor humano.

Importe de oferta sugerido: 6,000.00 €

Agente F · Asistente de conciliación

Agente F: expediente CLM-CONC-0001 (dpa, tramo 1). Subir al siguiente tramo (75% = 6,000.00 EUR). El Agente F solo recomienda: la ejecuta un gestor humano. Prioridad: baja. Sin alertas activas.

CLM-CONC-0002

DPA

4,500 € reclamados

Riesgo de cierre por abandono

Oferta al 65% ya hecha

Prioridad media

Recomendación: Esperar respuesta del cliente a la oferta vigente (65%). El Agente F solo recomienda: la ejecuta un gestor humano.

riesgo de cierre por abandono

Agente F · Asistente de conciliación

Agente F: expediente CLM-CONC-0002 (dpa, tramo 1). Esperar respuesta del cliente a la oferta vigente (65%). El Agente F solo recomienda: la ejecuta un gestor humano. Prioridad: media. Alertas activas: riesgo de cierre por abandono.

CLM-CONC-0003

RC

15,000 € reclamados

RC con cobertura insuficiente

Sin oferta formulada

Prioridad media

Recomendación: Formular oferta inicial (65% = 9,750.00 EUR). El Agente F solo recomienda: la ejecuta un gestor humano.

Importe de oferta sugerido: 9,750.00 €

riesgo de escalada judicial — revisar con legal

Agente F · Asistente de conciliación

Agente F: expediente CLM-CONC-0003 (rc, tramo 0). Formular oferta inicial (65% = 9,750.00 EUR). El Agente F solo recomienda: la ejecuta un gestor humano. Prioridad: media. Alertas activas: riesgo de escalada judicial — revisar con legal.

47 / 68

Figura 3.10. Demostrador del Agente F sobre tres expedientes en negociación: prioridad, recomendación, importe de oferta sugerido, alertas y razonamiento por reglas.

El Agente F (asesor de conciliación) **no forma parte del grafo A→B→C→G→D→E**: es un módulo independiente que recomienda el siguiente paso en negociaciones de daños propios (DPA) y responsabilidad civil (RC) tras un primer rechazo de oferta. Funciona **solo por reglas explicables**, sin LLM ni aprendizaje automático, y **nunca ejecuta** la acción: la recomendación la aplica un gestor humano.

Reglas que aplica (constantes de `backend/app/agents/conciliation_advisor.py`):

Situación	Recomendación	Importe de oferta
Sin oferta previa (tramo 0)	Formular oferta inicial	65 % del importe reclamado
Oferta del tramo 1 o 2 rechazada	Subir al siguiente tramo	75 % (tras el tramo 1) · 90 % (tras el tramo 2)
Oferta del tramo 3 (90 %) rechazada	Decisión humana requerida: transar una última vez, cierre por abandono o derivación a vía judicial; el Agente F no elige entre las tres	—
Oferta vigente, sin respuesta del cliente	Esperar respuesta del cliente a la oferta vigente	—

Alertas que puede emitir: *riesgo de cierre por abandono* (más de 30 días sin respuesta del cliente), *expediente estancado* — *priorizar* (más de 45 días en el tramo de negociación actual) y *riesgo de escalada judicial* — *revisar con legal* (expediente de RC cuya cobertura es insuficiente frente a la reclamación). La prioridad se deriva del número de alertas: **alta** (2 o más), **media** (1), **baja** (0). Cada recomendación termina con la nota «El Agente F solo recomienda: la ejecuta un gestor humano.»

Los tres expedientes de demostración (`streamlit_conciliation_fixtures.py`):

Expediente	Tipo / importe	Situación	Recomendación mostrada
CLM-CONC-0001	DPA · 8.000 €	Oferta del tramo 1 rechazada	Subir al siguiente tramo (75 % = 6.000,00 €)
CLM-CONC-0002	DPA · 4.500 €	Oferta del tramo 1 sin respuesta desde hace 40 días	Esperar respuesta; alerta de abandono; prioridad media
CLM-CONC-0003	RC · 15.000 €	Sin oferta previa; cobertura insuficiente	Formular oferta inicial (65 % = 9.750,00 €); alerta de escalada judicial

5.9 Vista Historial

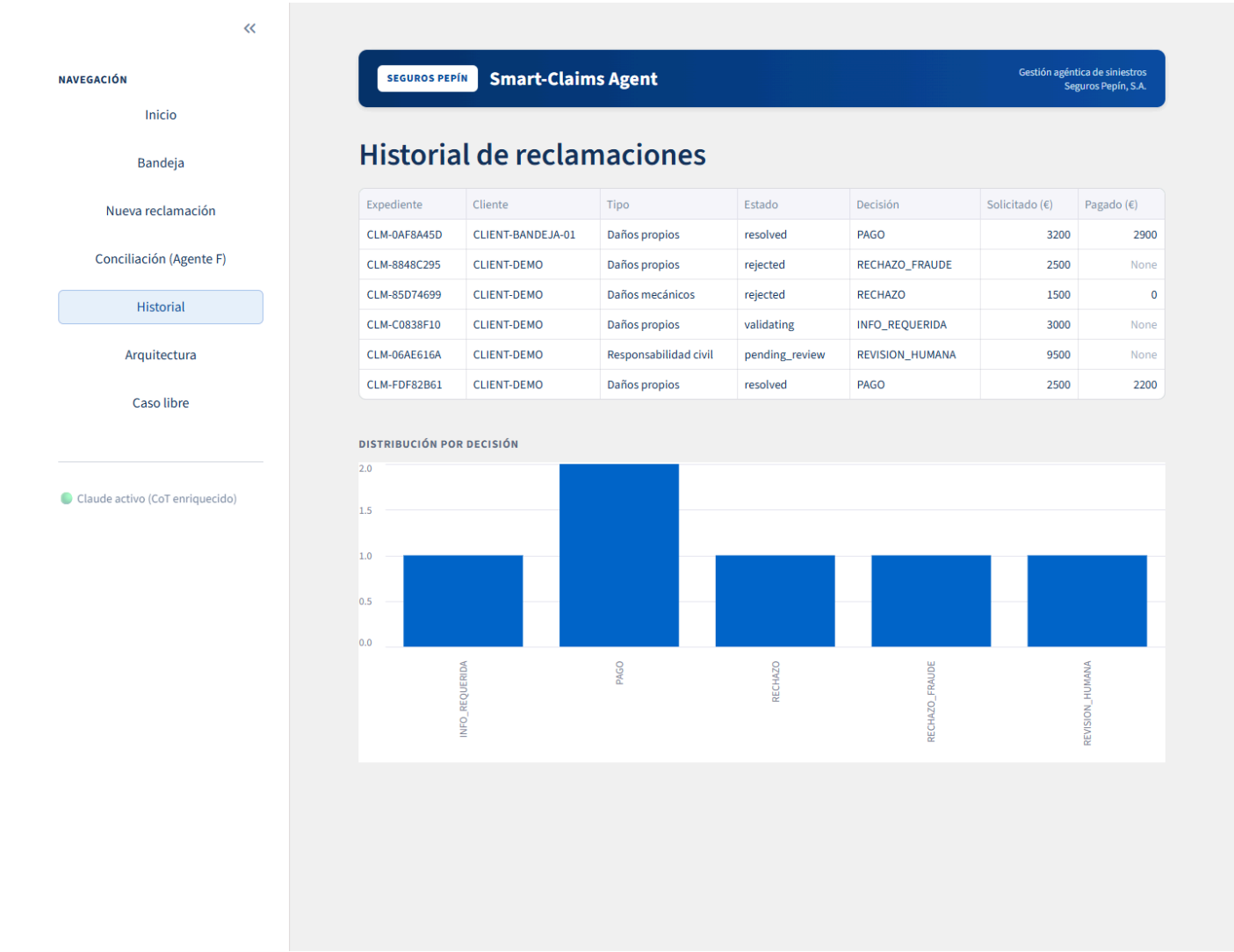


Figura 3.11. Historial de la sesión tras procesar los cinco escenarios y un ticket de la Bandeja: tabla de expedientes y distribución por decisión.

Tabla con los expedientes procesados **en la sesión actual del navegador** (columnas **Expediente**, **Cliente**, **Tipo**, **Estado**, **Decisión**, **Solicitado (€)**, **Pagado (€)**) y gráfico de barras con el recuento por decisión. Al recargar la página, la sesión de Streamlit se reinicia y el historial se vacía; la persistencia duradera corresponde a MariaDB (\$8), disponible solo en el Modo 1.

5.10 Vista Arquitectura

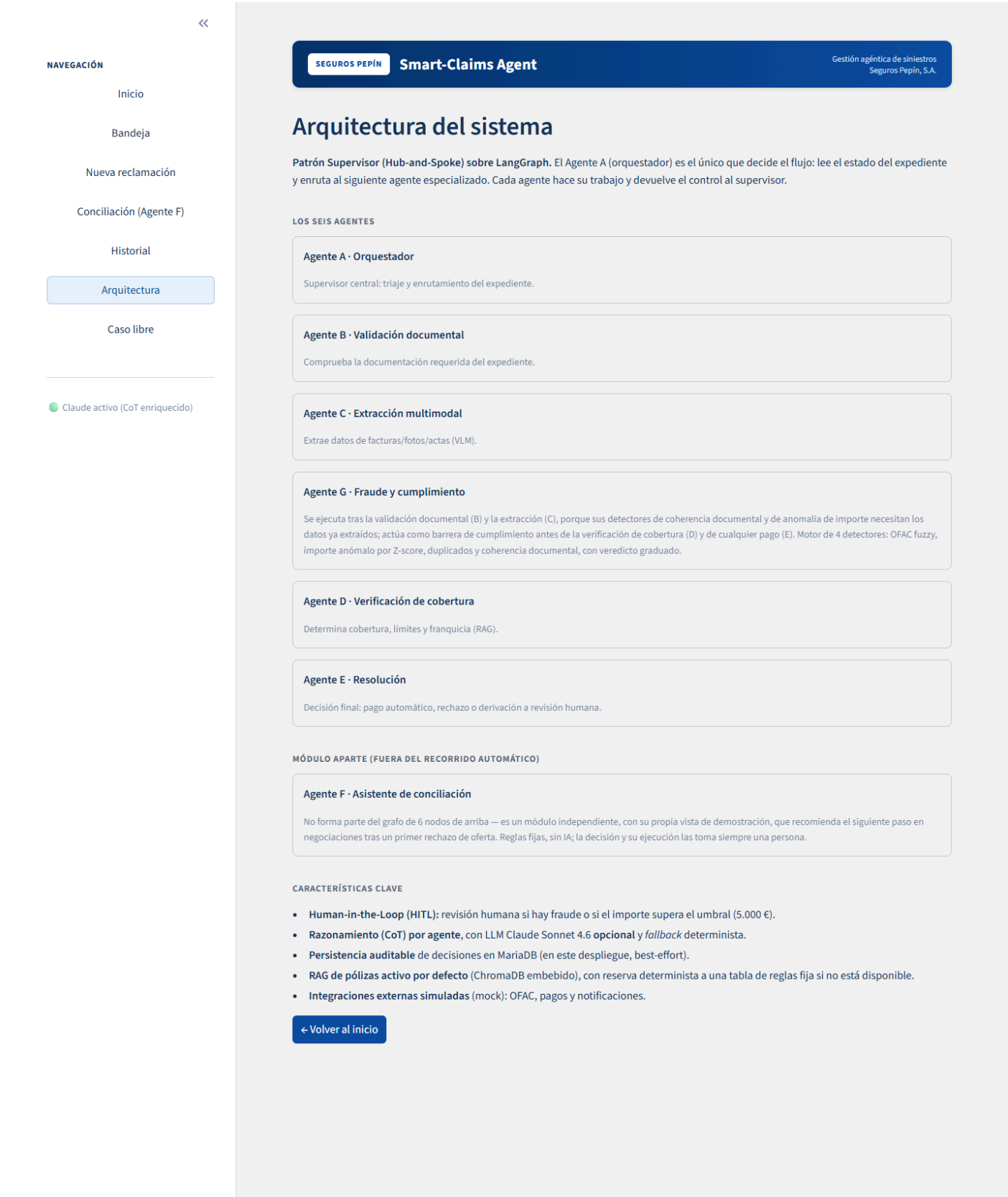
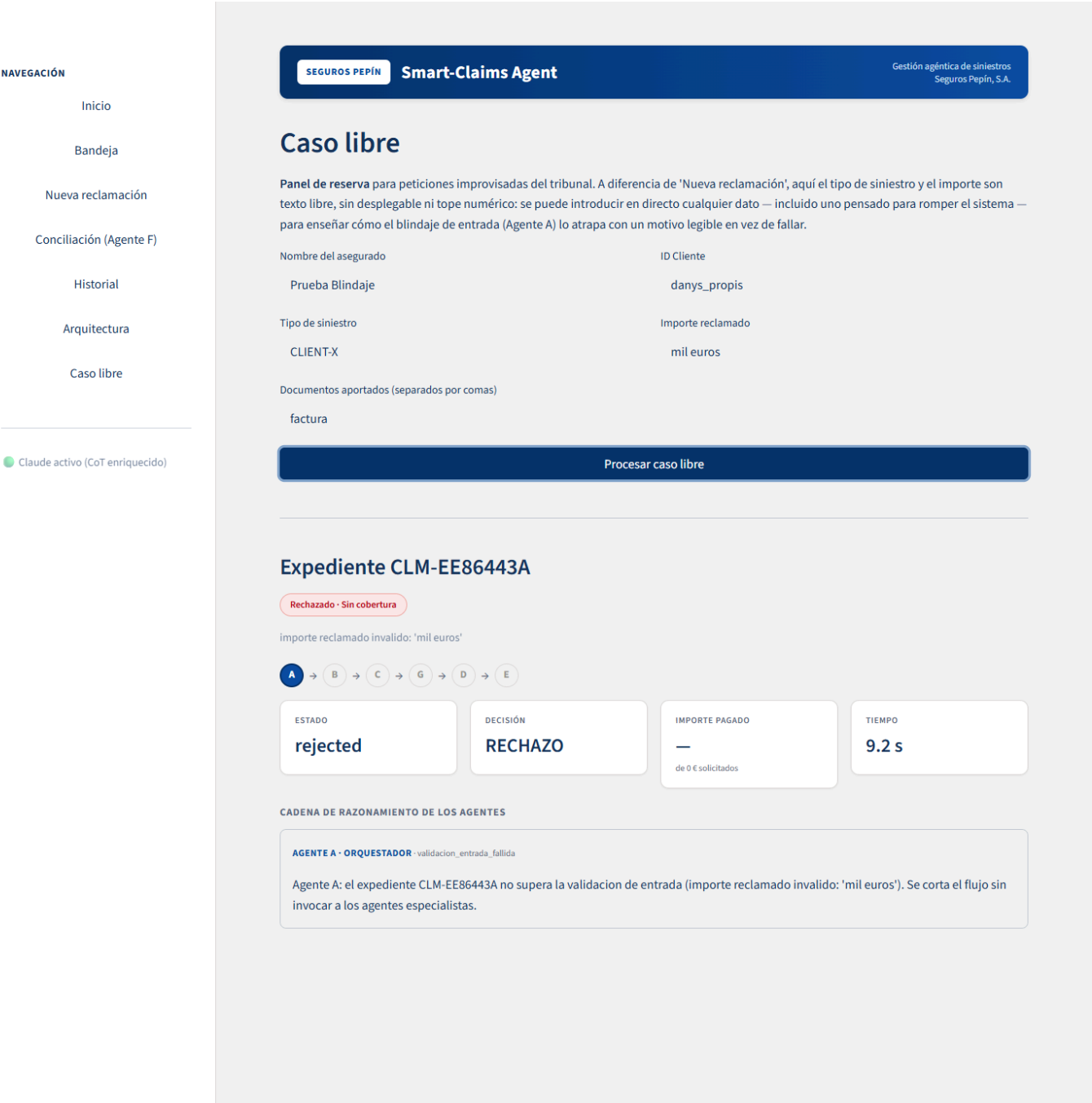


Figura 3.12. Vista Arquitectura: patrón Supervisor (Hub-and-Spoke) sobre LangGraph, los seis agentes en orden de ejecución, el Agente F como módulo aparte y las características clave.

Vista informativa para el tribunal: describe el patrón Supervisor, los seis agentes en su orden de ejecución (A, B, C, G, D, E) con la justificación de por qué el Agente G se ejecuta tras la extracción (sus detectores de coherencia documental y de anomalía de importe necesitan los datos ya extraídos) y antes de cobertura y pago, el Agente F como módulo independiente, y las características clave (HITL, CoT opcional, persistencia auditable, RAG activo por defecto, integraciones simuladas).

5.11 Vista Caso libre y blindaje de entrada



Pieza	Dónde	Qué hace	Qué se ve en pantalla
de entrada		revisión humana tipos de siniestro no reconocidos y nombres vacíos o de más de 200 caracteres; elimina documentos duplicados	invalido: 'mil euros'; REVISION_HUMANA con motivo tipo de siniestro no reconocido: ... o nombre del asegurado invalido (vacío o excesivamente largo)
A2 — captura global de errores	<code>process_claim</code> (orquestador)	Cualquier excepción interna no prevista se convierte en <code>REVISION_HUMANA</code> con motivo <code>error_interno_controlado</code>	Resultado normal con estado <code>pending_review</code> ; nunca un traceback
A3 — sin trazas en la interfaz	<code>streamlit_app.py</code>	Si aun así algo falla al procesar, la app muestra un mensaje breve y registra el incidente en el log del servidor	«No se pudo procesar la reclamación. Se ha registrado el incidente.»
A4 — validación en el formulario	Vista Nueva reclamación	El desplegable solo ofrece los cuatro tipos válidos y el importe está acotado a 0–100.000 €	Aviso si faltan tipo o importe
A5 — panel Caso libre	Vista Caso libre	Permite alcanzar desde la interfaz las ramas de A1 que el formulario acotado impide	Cualquier motivo de A1

Un tipo de siniestro desconocido (por ejemplo `inundacion`) no se rechaza: se deriva a un gestor (`REVISION_HUMANA`), ya que puede tratarse de un tipo real no contemplado por el catálogo sintético.

6. Modo 3 — CLI de demostración, API REST y evaluador

6.1 CLI de demostración

La CLI ejecuta cuatro expedientes predefinidos directamente sobre el orquestador Python y muestra el Chain of Thought y la decisión en la terminal. No requiere Docker, MariaDB ni ChromaDB.

Instalación de dependencias (una sola vez)

```
# Windows — desde la raíz del repositorio
py -m pip install -r backend/requirements.txt
```

```
# Linux / macOS
python3.11 -m pip install -r backend/requirements.txt
```

Ejecución

```
# Windows – desde la raíz del repositorio
py backend/scripts/run_demo.py
```

```
# Linux / macOS
python3.11 backend/scripts/run_demo.py
```

O desde el contenedor del backend (si Docker está levantado):

```
docker exec -it sca-backend python scripts/run_demo.py
```

Casos de demostración

El script ejecuta cuatro expedientes con una semilla aleatoria fija (`random.seed(7)`) para garantizar reproducibilidad:

Expediente	claim_type	Importe	Documentos	Decisión esperada
DEMO-PAGO	danys_propis	3.200 €	Completos	PAGO
DEMO-HITL	responsabilitat	8.500 €	Completos	REVISION_HUMANA
DEMO-RECHAZO	danys_mecanics	1.000 €	Completos	RECHAZO
DEMO-INFO	danys_propis	1.000 €	Solo factura	INFO_REQUERIDA

El camino `RECHAZO_FRAUDE` no está en la CLI: se demuestra desde la app Streamlit (escenario OFAC, \$5.4.1), porque depende del nombre del asegurado.

Ejemplo de salida

```
-----
Expediente: DEMO-PAGO
Escenario: Pago automatico (cobertura + importe bajo)
Tipo:      danys_propis | Importe: 3200.0 EUR
-----

Razonamiento (Chain of Thought):
  1. Agente A: expediente DEMO-PAGO de tipo 'danys_propis' por importe 3200.0 EUR. Se
  inicia el flujo de procesamiento con cribado antifraude como filtro de entrada.
  2. Agente B: documentacion completa y conforme. Documentos requeridos: foto_danys,
  factura, denuncia_companyia. Documentos faltantes: ninguno.
  3. Agente C: extraidos 3 documentos con confianza media 0.83. Atencion: baja
  confianza en ['foto_danys', 'factura', 'denuncia_companyia']. Importe inferido: 4305.77
  EUR.
  4. Agente G: veredicto CLEAR (score 0.00). Sin indicios relevantes. Senales
  activas: Sin senales de fraude detectadas.
  5. Agente D: cobertura según el catálogo de pólizas. Siniestro 'danys_propis'
  cubierto segun seccion SP-PCS-009 § 3.2. Importe neto pagable: 2900.00 EUR (limite
  10000 EUR, franquicia 300 EUR).
  6. Agente E: cobertura confirmada (seccion SP-PCS-009 § 3.2) e importe 2900.00 EUR
```

dentro del umbral. Se aprueba el PAGO automatico.

```
>>> Decision:  PAGO
      Estado:   resolved
      HITL:     False
      Importe pagado: 2900.0 EUR
```

Salida real del script en modo fallback (sin clave). El importe pagado (2.900 €) corresponde a los 3.200 € reclamados menos la franquicia de 300 € de la póliza de daños propios. Sin ficheros adjuntos, el Agente C simula la extracción (confianza e «importe inferido» generados con la semilla fija), lo que explica el aviso de baja confianza; ese importe inferido no interviene en la decisión.

6.2 API REST

Con el backend levantado (Docker o ejecución local), la API REST acepta peticiones en <http://localhost:8000>.

Endpoints disponibles

Método	Endpoint	Descripción
GET	/health	Estado del servicio ({"status":"ok","version":"0.5.0"})
GET	/api/v1/agents/status	Estado y descripción de los seis agentes
POST	/api/v1/claims/	Procesa un expediente → decisión + CoT + HITL
GET	/api/v1/claims/	Lista expedientes (paginación y filtro por estado)
GET	/api/v1/claims/{claim_id}	Detalle de un expediente con todas sus decisiones
GET	/api/v1/claims/{claim_id}/trace	Solo el Chain of Thought de un expediente

La documentación Swagger interactiva está en <http://localhost:8000/docs>. El cuerpo del POST admite `claim_id` (opcional; si se omite, el backend genera uno), `client_id`, `client_email`, `claim_type`, `channel`, `amount_requested`, `documents` y `text`. El nombre del asegurado y los ficheros adjuntos solo se pueden aportar desde la app Streamlit.

Ejemplo 1: PAGO (daños propios, importe bajo, docs completos)

```
curl -s -X POST http://localhost:8000/api/v1/claims/ \
-H "Content-Type: application/json" \
-d '{
  "claim_id": "CLM-PAGO-01",
  "client_id": "C-A",
  "claim_type": "danys_propis",
  "channel": "email",
  "amount_requested": 3200.0,
  "documents": ["foto_danys", "factura", "denuncia_companyia"]
}'
```

Respuesta esperada (resumen):

```
{
  "claim_id": "CLM-PAGO-01",
  "status": "resolved",
  "decision": "PAGO",
  "amount_paid": 2900.0,
  "hitl_required": false
}
```

Ejemplo 2: REVISION_HUMANA (importe supera el umbral HITL)

```
curl -s -X POST http://localhost:8000/api/v1/claims/ \
-H "Content-Type: application/json" \
-d '{
  "claim_id": "CLM-HITL-01",
  "client_id": "C-B",
  "claim_type": "responsabilitat",
  "channel": "web",
  "amount_requested": 8500.0,
  "documents": ["foto_danys", "acta_policial", "dades_tercer"]
}'
```

Respuesta esperada (resumen):

```
{
  "claim_id": "CLM-HITL-01",
  "status": "pending_review",
  "decision": "REVISION_HUMANA",
  "amount_paid": null,
  "hitl_required": true,
  "termination_reason": "importe 8500.0 EUR supera umbral HITL (5000.0 EUR)"
}
```

Ejemplo 3: RECHAZO (tipo sin cobertura)

```
curl -s -X POST http://localhost:8000/api/v1/claims/ \
-H "Content-Type: application/json" \
-d '{
  "claim_id": "CLM-RECH-01",
  "client_id": "C-C",
  "claim_type": "danys_mecanics",
  "channel": "email",
  "amount_requested": 1000.0,
  "documents": ["informe_taller", "factura"]
}'
```

Respuesta esperada (resumen):

```
{
  "claim_id": "CLM-RECH-01",
  "status": "rejected",
  "decision": "RECHAZO",
  "hitl_required": false,
  "termination_reason": "rechazado por no cobertura"
}
```

Ejemplo 4: INFO_REQUERIDA (documentación incompleta)

```
curl -s -X POST http://localhost:8000/api/v1/claims/ \
-H "Content-Type: application/json" \
-d '{
  "claim_id": "CLM-INFO-01",
  "client_id": "C-D",
  "claim_type": "danys_propis",
  "channel": "email",
  "amount_requested": 1000.0,
  "documents": ["factura"]
}'
```

Respuesta esperada (resumen):

```
{
  "claim_id": "CLM-INFO-01",
  "status": "validating",
  "decision": "INFO_REQUERIDA",
  "hitl_required": false,
  "termination_reason": "documentacion incompleta: faltan foto_danys,
denuncia_companyia"
}
```

Ejemplo 5: importe anómalo (MEDIUM_RISK, sin bloqueo)

La API no admite el nombre del asegurado, por lo que el bloqueo **RECHAZO_FRAUDE** por lista de sanciones solo se demuestra desde la app Streamlit (§5.4.1). Sí puede observarse el detector de importe anómalo: un importe por encima del máximo histórico del tipo (9.000 € para **danys_propis**) suma +0,40 al riesgo, lo que produce el veredicto **MEDIUM_RISK** sin bloquear el expediente, que continúa hasta el Agente E y termina en revisión humana por importe:

```
curl -s -X POST http://localhost:8000/api/v1/claims/ \
-H "Content-Type: application/json" \
-d '{
  "claim_id": "CLM-ANOM-01",
  "client_id": "C-Z",
  "claim_type": "danys_propis",
  "channel": "email",
  "amount_requested": 9800.0,
}
```

```
"documents": ["foto_danys", "factura", "denuncia_companyia"]
}'
```

Respuesta esperada (resumen):

```
{
  "claim_id": "CLM-ANOM-01",
  "status": "pending_review",
  "decision": "REVISION_HUMANA",
  "hitl_required": true,
  "termination_reason": "importe 9500.0 EUR supera umbral HITL (5000.0 EUR)"
}
```

En la traza (GET /api/v1/claims/CLM-ANOM-01/trace), la entrada del Agente G indica veredicto MEDIUM_RISK (score 0.40) con la señal de importe anómalo (z-score 5,0; máximo histórico superado).

Consultar un expediente ya procesado

```
curl -s http://localhost:8000/api/v1/claims/CLM-PAGO-01
```

Si el expediente no existe en la base de datos, la respuesta es HTTP 404.

6.3 Evaluador in-process y tests automáticos

Para reproducir las métricas del capítulo 4 sin backend ni base de datos, el evaluador ejecuta los 32 casos del dataset sintético directamente sobre el orquestador (retira la clave de Anthropic del entorno para que el resultado sea determinista) y escribe data/synthetic/evaluation_inprocess.json:

```
# Windows – desde la carpeta backend/
cd backend
py scripts/evaluate_inprocess.py
```

La batería de pruebas automáticas (100 tests: unitarios por agente, orquestación, blindaje, Agente F, fixtures de la app y contrato de la interfaz) se ejecuta con:

```
cd backend
py -m pytest tests -q
```

Los tests no requieren clave de Anthropic ni servicios externos.

7. Interpretación de resultados

7.1 Decisiones posibles

decision	status	Píldora en la app	Significado	Agente que la produce
PAGO	resolved	Resuelto · Pago aprobado	Cobertura confirmada, sin fraude, importe neto dentro del umbral; pago simulado al IBAN de prueba	E
REVISION_HUMANA	pending_review	Revisión humana requerida	Importe neto superior al umbral HITL, tipo no reconocido, nombre inválido o error interno controlado	E (o A en el blindaje)
INFO_REQUERIDA	validating	Información requerida	Faltan documentos obligatorios; se simula una solicitud al cliente	B
RECHAZO	rejected	Rechazado · Sin cobertura	Tipo excluido de la póliza, o importe inválido (blindaje A1)	D (o A en el blindaje)
RECHAZO_FRAUDE	rejected	Bloqueado · Fraude / OFAC	Veredicto HIGH_RISK o BLOCKED del Agente G; el expediente pasa a compliance	G

7.2 Human-in-the-Loop (HITL) y el campo **hitl_required**

El Agente E deriva a revisión humana cuando el **importe neto a pagar** (importe reclamado, limitado al máximo de la póliza, menos la franquicia) es **estrictamente superior** al umbral **HITL_AMOUNT_THRESHOLD** (5.000 € por defecto). Un importe neto de exactamente 5.000 € se paga automáticamente. Ejemplos con el umbral por defecto:

Tipo	Importe reclamado	Importe neto	Decisión
danys_propis	5.300 €	5.000 €	PAGO
danys_propis	5.400 €	5.100 €	REVISION_HUMANA
responsabilitat	5.000 €	5.000 €	PAGO
responsabilitat	5.001 €	5.001 €	REVISION_HUMANA

El campo **hitl_required** es **true** en **REVISION_HUMANA** y en **RECHAZO_FRAUDE** (un gestor de compliance debe revisar el bloqueo). En el prototipo, la derivación termina el flujo; la revisión humana posterior queda fuera del alcance del MVP (la tabla **hitl_feedback** de MariaDB está preparada para registrarla).

7.3 Veredicto de fraude (Agente G)

El Agente G ejecuta cuatro detectores deterministas sobre datos sintéticos y combina sus señales en una puntuación de riesgo 0–1:

Detector	Señal	Umbral / regla	Aportación a la puntuación
Lista de sanciones (OFAC/SDN sintética)	Similitud difusa entre el nombre del asegurado y las 15 entradas de la lista (p. ej. <i>Viktor Nikolaev Kozlov</i> , <i>Amira Belhaj</i> , <i>Al-Rashid Trading Group</i>)	Similitud $\geq$ 0,82	Puntuación 1,0 y veredicto BLOCKED directo

Detector	Señal	Umbral / regla	Aportación a la puntuación
Importe anómalo	Z-score del importe frente al <i>baseline</i> del tipo (danys_propis : media 2.800 €, desv. 1.400 €, máx. 9.000 €; responsabilitat : 12.000 / 8.000 / 48.000; robatori : 3.200 / 1.600 / 7.500; danys_mecanics : 800 / 400 / 3.000)	$ z \geq 2,0$, o importe > máximo histórico	+0,40 si supera el máximo; si no, +min($ z /5$, 0,35)
Duplicados	Siniestro del mismo cliente y tipo en los últimos 90 días (historial simulado: C-A daños propios 20-05-2026, C-B RC 15-04-2026, C-C robo 10-03-2026)	Ventana de 90 días	+0,35 (×0,65 si el anterior tiene más de 30 días)
Coherencia documental	Discrepancias entre lo declarado y lo extraído por el Agente C (importes, fechas, tipo de documento)	Cada incidencia	+0,10 por incidencia, máximo +0,25

Veredicto: **BLOCKED** (coincidencia OFAC), **HIGH_RISK** ($\geq 0,55$), **MEDIUM_RISK** ($\geq 0,25$) o **CLEAR**. Solo **BLOCKED** y **HIGH_RISK** bloquean el expediente (**RECHAZO_FRAUDE**); **MEDIUM_RISK** se registra en el razonamiento y el flujo continúa hacia el Agente D.

Las fechas del historial simulado son fijas, por lo que la ventana de 90 días depende de la fecha del sistema: pasados 90 días desde esas fechas, el detector de duplicados deja de activarse para **C-A**, **C-B** y **C-C**.

7.4 Cobertura RAG (Agente D)

El resultado de cobertura contiene **covered**, **max_amount**, **deductible**, **policy_section**, **source** (**rag** o **mock**), **retrieved_snippet** y **retrieval_distance**. Con **source** = "**rag**", la sección y el fragmento proceden de la búsqueda semántica sobre **data/policies/**; con **source** = "**mock**", del catálogo determinista de **check_policy** (tabla de §5.4.3). En ambos casos el importe neto se calcula como $\max(0, \min(\text{importe}, \text{máximo}) - \text{franquicia})$.

7.5 Extracción multimodal (Agente C · Claude Vision)

- **Con ficheros subidos y clave de Anthropic:** cada fichero (imagen o PDF) se envía a **claude-sonnet-4-6** con una instrucción de extracción estructurada; el resultado tiene **doc_type**, **amount**, **date**, **vendor**, **summary** y **confidence**, y la fuente es **claude_vision**. Una confianza inferior a 0,85 se marca como baja en el razonamiento.
- **Con ficheros subidos pero sin clave:** el documento aparece como **desconocido** con el resumen «Extracción no disponible (sin clave LLM o error).» y confianza 0; el flujo continúa.
- **Sin ficheros:** el Agente C simula la extracción a partir de los tipos de documento declarados (fuente **mock**).

El importe que decide el expediente es siempre el **declarado** en el formulario; el importe leído por Vision se usa para el detector de coherencia del Agente G y se muestra a título informativo.

7.6 Chain of Thought (**reasoning_trace** / **decisions_log**)

Cada agente añade una entrada a **decisions_log** (agente, acción, razonamiento, confianza, **hitl_required**) y una línea a **reasoning_trace**. En la app, la sección «Cadena de razonamiento de los agentes» muestra **decisions_log** como una línea de tiempo; en la API, **GET /api/v1/claims/{id}/trace** devuelve la misma información. Con clave, el texto lo redacta Claude a partir de los datos deterministas del agente; sin clave, es una plantilla de fallback con los mismos datos.

8. Inspección de la base de datos con Adminer

8.1 Acceso a Adminer

Con el sistema Docker levantado, abrir en el navegador:

```
http://localhost:8082
```

Introducir los siguientes datos de conexión:

Campo	Valor
Sistema	MariaDB
Servidor	<code>mariadb</code>
Usuario	<code>claims_user</code>
Contraseña	<code>claims_secret_dev</code> (o el valor configurado en <code>.env</code>)
Base de datos	<code>smart_claims</code>

8.2 Tablas del esquema

La base de datos `smart_claims` contiene tres tablas:

Tabla	Descripción
<code>claims</code>	Un registro por expediente. Columnas principales: <code>id</code> , <code>client_id</code> , <code>claim_type</code> , <code>channel</code> , <code>status</code> , <code>amount_requested</code> , <code>amount_approved</code> , <code>created_at</code> .
<code>agent_decisions</code>	Una fila por decisión de cada agente. Columnas: <code>claim_id</code> (FK), <code>agent</code> , <code>action</code> , <code>reasoning</code> (texto completo del CoT), <code>confidence</code> , <code>hitl_required</code> , <code>created_at</code> .
<code>hitl_feedback</code>	Preparada para registrar el feedback del operador humano en casos HITL. Columnas: <code>claim_id</code> , <code>decision_id</code> (FK), <code>reviewer</code> , <code>original_action</code> , <code>final_action</code> , <code>override_reason</code> . En el MVP actual está vacía; se alimentará en fases posteriores.

8.3 Consultas SQL útiles

Traza completa de decisiones de un expediente:

```
SELECT
  ad.created_at,
  ad.agent,
  ad.action,
  ad.reasoning,
  ad.confidence,
  ad.hitl_required
FROM agent_decisions ad
WHERE ad.claim_id = 'CLM-PAGO-01'
ORDER BY ad.id ASC;
```

Estado final de un expediente:

```
SELECT id, claim_type, status, amount_requested, amount_approved, created_at
FROM claims
WHERE id = 'CLM-PAGO-01';
```

Resumen de expedientes por estado:

```
SELECT status, COUNT(*) AS total
FROM claims
GROUP BY status
ORDER BY total DESC;
```

9. Resolución de problemas frecuentes

9.1 Sin `ANTHROPIC_API_KEY` — el sistema usa el fallback determinista

Síntoma: el razonamiento en «Cadena de razonamiento» es breve y esquemático; en la barra lateral aparece  `Modo fallback determinista (sin clave)`; en la Bandeja, los adjuntos se muestran como `desconocido` con «Extracción no disponible».

Causa: la variable `ANTHROPIC_API_KEY` no está configurada o es inválida.

Solución: añadir una clave válida de Anthropic en `.env` y reiniciar (`docker compose restart backend` en el Modo 1; relanzar `streamlit run` en el Modo 2). En Streamlit Cloud, añadir la clave en la sección `Secrets` del panel de administración y reiniciar la app.

El comportamiento de la demo es correcto en cualquier caso; el fallback es un comportamiento previsto del diseño.

9.2 Sin MariaDB — la CLI y la app muestran un aviso en el log pero continúan

Síntoma (CLI o consola de Streamlit): aparece una línea de log similar a:

```
WARNING root: No se han podido persistir las decisiones de DEMO-PAGO: ...
```

Causa: se ejecuta sin el servicio MariaDB levantado. `process_claim` captura la excepción y continúa el flujo sin interrupciones; en la app, el usuario no ve ningún error.

Solución: este comportamiento es intencional. Para persistencia completa, usar el despliegue Docker (\$4).

9.3 El Agente D muestra `source = "mock"` (sin sección RAG en el resultado)

Síntoma: en el resultado no aparece la sección «Cobertura (Agente D · RAG sobre pólizas)».

Causa: `SCA_RAG_ENABLED` está vacío o a `0`, o la colección embebida no pudo indexarse (por ejemplo, falta la carpeta `data/policies/`). El Agente D cae automáticamente al catálogo determinista.

Solución: comprobar `SCA_RAG_ENABLED=1` y que `data/policies/` contiene los cuatro ficheros `SP-PCS-009-*.md`. La decisión es la misma en ambos modos; solo cambia el origen de la sección de póliza citada.

9.4 Puerto ocupado al arrancar Docker o Streamlit

Síntoma (Docker):

```
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:8000 ->
...
```

Síntoma (Streamlit): Port 8501 is already in use.

Causa: uno de los puertos requeridos (8000, 8080, 8082, 3306 o 8501) está en uso, a menudo por otra instancia de la propia app.

Solución en Windows:

```
netstat -ano | findstr :8501
taskkill /PID <PID> /F
```

Alternativamente, cambiar el puerto (`streamlit run streamlit_app.py --server.port 8600`) o el mapeo en `docker-compose.yml` (columna izquierda del par `host:contenedor`).

9.5 El backend no arranca (`sca-backend` en estado `restarting`)

Causa más frecuente: MariaDB no ha completado su inicialización cuando el backend intenta conectarse. El `docker-compose.yml` ya define la condición `service_healthy` para el healthcheck de MariaDB, pero en equipos lentos puede necesitar más tiempo.

Solución: esperar entre 30 y 60 segundos y verificar:

```
docker compose ps
docker compose logs backend --tail=30
```

Si el problema persiste, comprobar que los valores `DB_*` en `.env` coinciden con los definidos en el bloque `mariadb` de `docker-compose.yml`.

9.6 `docker compose up` no levanta ningún frontend en el puerto 8501

Causa: el servicio `frontend` está comentado en `docker-compose.yml` desde agosto de 2026 (§4.2); el dashboard vigente es `streamlit_app.py`.

Solución: lanzar la app por separado con `py -m streamlit run streamlit_app.py` (§5.1). No es necesario descomentar el bloque.

9.7 Error 404 Not Found al consultar `GET /api/v1/claims/{id}`

Causa: el expediente no existe en la base de datos. Esto ocurre cuando se usa la CLI o la app Streamlit sin MariaDB disponible, o cuando el `claim_id` de la consulta no coincide con el que usó `process_claim`.

Solución: enviar primero el expediente con `POST /api/v1/claims/` con el sistema Docker activo, y consultar inmediatamente después con el mismo `claim_id`.

9.8 El Agente G bloquea un expediente de prueba que parecía legítimo

Causa: el motor antifraude es determinista (§7.3). El bloqueo (**BLOCKED**) se debe casi siempre a que el nombre del asegurado se parece (similitud $\geq 0,82$) a una entrada de la lista de sanciones sintética; el veredicto **HIGH_RISK** requiere acumular al menos 0,55 entre importe anómalo (máximo +0,40), duplicado reciente del mismo cliente (+0,35, solo para **C-A**, **C-B** y **C-C** dentro de la ventana de 90 días) e incoherencias documentales (máximo +0,25). Un importe anómalo por sí solo da **MEDIUM_RISK** (0,40), que no bloquea.

Solución: la sección «Cribado antifraude» del resultado enumera las señales activadas. Para una prueba limpia, usar un nombre corriente, un ID distinto de **C-A/C-B/C-C** y un importe dentro del rango habitual del tipo.

9.9 El resultado desaparece al cambiar de vista

Causa: comportamiento deliberado (§5.2): la navegación descarta **last_result** para que no se muestre un expediente antiguo en otra vista.

Solución: consultar el expediente en *Historial*, o volver a procesarlo.

9.10 El formulario no admite importes superiores a 100.000 € ni tipos fuera del desplegable

Causa: el formulario de *Nueva reclamación* acota deliberadamente las entradas (blindaje A4, §5.11).

Solución: usar la vista *Caso libre*, cuyos campos de texto libre permiten cualquier valor y muestran cómo el blindaje A1 lo trata (rechazo por importe inválido, revisión humana por tipo no reconocido, etc.).

9.11 **RuntimeError: Event loop is closed** al terminar la CLI

Síntoma: la CLI termina con un traceback cosmético:

```
Exception ignored in: <function Connection.__del__ ...>
RuntimeError: Event loop is closed
```

Causa: el driver **aiomysql** intenta cerrar sus conexiones después de que el bucle asíncrono se ha cerrado. No afecta al resultado del flujo; es un aviso puramente cosmético.

Solución: ignorar el aviso. El script **run_demo.py** ya incluye **await engine.dispose()** al final de **main()** para minimizar este comportamiento.

9.12 La barra lateral indica «Claude activo» pero el razonamiento es el del fallback

Síntoma: el indicador muestra  **Claude activo (CoT enriquecido)**, pero las tarjetas de razonamiento contienen la línea determinista breve y, en la Bandeja, los adjuntos aparecen con «Extracción no disponible (sin clave LLM o error)» y confianza 0 %. En el log del proceso (**streamlit run** o **docker compose logs backend**) aparecen líneas como:

```
Fallback de razonamiento (LLM no disponible): Error code: 400 - {... 'message': 'Your
credit balance is too low to access the Anthropic API. ...'}
Extracción multimodal con Claude falló (factura_taller_can_bosch.pdf): Error code: 400
- {...}
```

Causa: el indicador de la barra lateral solo comprueba que `ANTHROPIC_API_KEY` exista (`streamlit_app.py`, `has_key = bool(os.getenv("ANTHROPIC_API_KEY"))`); no valida la clave ni el saldo. Si la clave es inválida, ha sido revocada o la cuenta no tiene crédito, cada llamada a la API devuelve un error 4xx y el sistema aplica el fallback previsto (§3.3): la decisión sigue siendo correcta, pero sin CoT enriquecido ni extracción multimodal.

Solución: revisar el saldo o la validez de la clave en la consola de Anthropic (*Plans & Billing*), corregirla en `.env` (o en *Secrets*, en Streamlit Cloud) y relanzar la app. Para una demo ante tribunal conviene procesar un expediente de prueba unos minutos antes y comprobar que la cadena de razonamiento muestra el texto extenso de la Figura 3.14.

10. Referencias

Amershi, S., Weld, D., Vorvoreanu, M., Fournay, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., y Horvitz, E. (2019). Software engineering for machine learning: A case study. *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*, 291–300.

<https://doi.org/10.1109/ICSE-SEIP.2019.00042>

Anthropic. (2024). *Claude API documentation*. <https://docs.anthropic.com>

FastAPI. (2024). *FastAPI documentation: Interactive API docs*. <https://fastapi.tiangolo.com/features/>

LangChain. (2025). *LangGraph documentation*. <https://langchain-ai.github.io/langgraph/>

Snowflake Inc. (2025). *Streamlit documentation*. <https://docs.streamlit.io>

Vrána, J. (2024). *Adminer — Database management in a single PHP file*. <https://www.adminer.org>

ewpage

Capítulo 4 — Evaluación y resultados

4. Evaluación y resultados

4.1 Objetivo de la evaluación

La evaluación del prototipo Smart-Claims Agent persigue dos objetivos:

1. **Validar el comportamiento funcional** del sistema agéntico sobre un conjunto representativo de expedientes, comprobando que ante cada reclamación produce la decisión correcta (pago automático, rechazo, revisión humana, solicitud de información o bloqueo por fraude) siguiendo el flujo descrito en el capítulo de arquitectura.
2. **Verificar las capacidades reales de IA** incorporadas al sistema: el motor antifraude de cuatro detectores (incluida la verificación OFAC), la recuperación aumentada (RAG) de cobertura y la lógica de resolución.

La evaluación se realiza sobre un **dataset sintético** generado expresamente, ya que no se dispone de acceso a expedientes reales de Seguros Pepín por la restricción de confidencialidad descrita en el capítulo 1. El uso de datos sintéticos es una práctica habitual en la validación temprana de sistemas de IA en dominios sensibles (Russell & Norvig, 2021).

4.2 Diseño del dataset sintético

4.2.1 Tamaño y composición

El dataset consta de **32 expedientes** estratificados en seis escenarios que cubren las cinco salidas del flujo más una demostración explícita del cribado OFAC:

Escenario	Casos	Decisión esperada
Pago automático	10	PAGO
Revisión humana por importe (HITL)	6	REVISION_HUMANA
Rechazo por no cobertura	4	RECHAZO
Documentación incompleta	8	INFO_REQUERIDA
Bloqueo por sanción OFAC	2	RECHAZO_FRAUDE
Importe inusual (potencial fraude)	2	REVISION_HUMANA
Total	32	

La estratificación refleja la distribución previsible de un escenario asegurador real, donde la mayoría de expedientes son pagos automáticos sobre importes modestos y una proporción menor requiere intervención humana, rechazo o bloqueo.

4.2.2 Generación

Los casos se construyen de forma reproducible (semilla fija de aleatoriedad). Para cada caso se fijan `claim_type`, `amount_requested`, `documents` y `client_name` con valores que activan el escenario objetivo:

- **Pago automático:** tipo cubierto (`dany_propis`, `responsabilitat`, `robatori`), importe por debajo del umbral HITL (5.000 €) y documentación completa.
- **Revisión humana:** importe entre 6.500 € y 40.000 € (por encima del umbral).
- **No cobertura:** tipo `dany_mecanics`, excluido por la póliza.
- **Documentación incompleta:** se elimina uno de los documentos requeridos.
- **Bloqueo OFAC:** se emplea como nombre del asegurado una entidad de la lista de sanciones sintética (p. ej. *Viktor Nikolaev Kozlov*, *Dmitri Volkov*).
- **Importe inusual:** importes anómalos (9.999,99 € y 7.777,77 €) con cliente sospechoso.

Las pólizas, la lista OFAC y los baselines de importe son **sintéticos** (placeholder del prototipo); en producción se alimentan con los datos reales de Seguros Pepín.

4.3 Protocolo de evaluación

La evaluación se ejecuta con el script `backend/scripts/evaluate_inprocess.py`, que invoca la función `process_claim` en proceso (sin necesidad del backend REST ni de MariaDB) sobre los 32 casos y compara la decisión obtenida con la esperada. Se eligió la evaluación en proceso porque:

1. No requiere levantar la infraestructura Docker completa, facilitando la reproducibilidad desde cualquier máquina.
2. Emplea el vocabulario de decisiones actual del sistema (`PAGO`, `RECHAZO`, `REVISION_HUMANA`, `INFO_REQUERIDA`, `RECHAZO_FRAUDE`).

El proyecto conserva además el evaluador `evaluate_dataset.py`, que ejecuta el mismo protocolo contra el endpoint REST cuando el sistema corre en Docker.

El protocolo se ejecuta con el **RAG real activo** (`SCA_RAG_ENABLED=1`) y **sin LLM externo** (se usa el *fallback* determinista del razonamiento), de modo que la evaluación es **reproducible** y **gratuita** —dos propiedades imprescindibles para que pueda validarse desde la propia máquina del tribunal—. La persistencia en MariaDB es *best-effort*: en el entorno de evaluación, sin base de datos, se omite con un aviso y el flujo continúa.

La evaluación se ejecuta deliberadamente **con el LLM externo desactivado**. Esto es posible porque, por diseño, la **decisión** del sistema es **determinista**: la toman las reglas (validación, motor antifraude de cuatro detectores, cobertura por RAG y umbral HITL), no el modelo generativo. El papel del LLM (Claude) es **enriquecer la traza de razonamiento (CoT)** y realizar la **extracción multimodal**, capacidades que se validan por separado (véase §4.8). Esta separación garantiza que las decisiones sean reproducibles y auditables, independientes de la variabilidad del modelo generativo.

4.4 Resultados globales

El objetivo de esta evaluación es validar la **corrección de la lógica del flujo** end-to-end sobre casos representativos, no estimar el rendimiento en producción. Sobre un dataset sintético y determinista, el sistema produce la decisión esperada en todos los casos; este resultado debe leerse como evidencia de que la lógica de decisión (validación → extracción → fraude → cobertura → resolución) es correcta y reproducible, no como una métrica de precisión generalizable (véase §4.10).

Métrica	Valor
Casos correctos	32 / 32
Precisión global	100 %
Tasa de Resolución Autónoma (TRA)	50,0 % (casos resueltos/rechazados sin HITL)
Tasa de HITL	25,0 %
Cobertura decidida por RAG real	68,8 % de los casos (los que alcanzan al Agente D)

El sistema clasificó correctamente los 32 expedientes. La **TRA del 50 %** corresponde a los pagos automáticos (10) y rechazos (4 por no cobertura + 2 por OFAC) que se resuelven sin intervención humana; el **25 % de HITL** corresponde a las 8 derivaciones a revisión humana por importe. La **cobertura se decidió por RAG** en el 68,8 % de los casos: el resto no llega al Agente D porque se corta antes (documentación incompleta → solicitud de información; sanción OFAC → bloqueo).

Latencia. El procesamiento agéntico real es **sub-segundo**: una media de **~0,25 s** por caso (mediana ~0,33 s, máximo ~0,41 s), medido con la persistencia aislada y el RAG activo. Los casos que recorren el flujo completo con recuperación RAG tardan ~0,3–0,4 s; los que se cortan antes (documentación incompleta, bloqueo OFAC) se resuelven en milisegundos. Los ~10 s observados en una ejecución previa correspondían íntegramente al *timeout* de conexión a MariaDB en un entorno sin base de datos (persistencia *best-effort*), no al razonamiento del sistema.

4.5 Resultados por escenario

Escenario	Aciertos	Total	Precisión
Pago automático	10	10	100 %
Revisión humana por importe	6	6	100 %
Rechazo por no cobertura	4	4	100 %

Escenario	Aciertos	Total	Precisión
Documentación incompleta	8	8	100 %
Bloqueo por sanción OFAC	2	2	100 %
Importe inusual (potencial fraude)	2	2	100 %
Global	32	32	100 %

4.6 Matriz de confusión

Filas = escenario esperado, columnas = decisión obtenida. La diagonal concentra los 32 aciertos; no hay ninguna confusión entre clases.

Esperado \ Obtenido	PAGO	REVISION_HUMANA	RECHAZO	INFO_REQUERIDA	RECHAZO_FRAUDE
Pago automático	10	0	0	0	0
Revisión por importe	0	6	0	0	0
No cobertura	0	0	4	0	0
Doc. incompleta	0	0	0	8	0
Sanción OFAC	0	0	0	0	2
Importe inusual	0	2	0	0	0

4.7 Validación del motor antifraude (Agente G)

La evaluación confirma el funcionamiento de los detectores reales:

- **Verificación OFAC/ONU:** los 2 expedientes con nombre sancionado se marcaron con veredicto **BLOCKED** y se resolvieron como **RECHAZO_FRAUDE**, demostrando que el *fuzzy matching* (umbral 0,82) contra la lista de sanciones funciona de extremo a extremo. El bloqueo automático por OFAC se justifica como obligación legal vinculante (no discrecional), a diferencia de la revisión humana por importe; podría derivarse a un oficial de cumplimiento como refinamiento.
- **Anomalía de importe (Z-score):** varios casos de importe elevado obtuvieron veredicto **MEDIUM_RISK** (el detector se activa, pero el score no alcanza el umbral de bloqueo de 0,55). Es el comportamiento correcto: el sistema señala la anomalía sin bloquear, y la derivación a revisión humana la produce el umbral de importe en el Agente E.
- **Coherencia documental:** verificado adicionalmente con pruebas unitarias (factura previa al siniestro → señal **factura_previa_al_siniestro**; fechas coherentes → sin señal).

A diferencia de la versión anterior del prototipo —que usaba un *mock* de fraude con un *score* aleatorio y producía algún falso positivo no determinista—, el motor actual es **determinista y auditable**, lo que elimina esa fuente de variabilidad y permite una evaluación reproducible.

4.8 Validación complementaria

- **Suite de tests automatizados: 100 tests** (pytest, SQLite en memoria) que cubren los agentes, la orquestación end-to-end, las herramientas, el motor antifraude, el RAG, la coherencia documental, el blindaje de entrada (A1–A4), el determinismo del núcleo de decisión frente al LLM, el asistente de conciliación (Agente F) y la

interfaz Streamlit. Se ejecutan sin MariaDB ni Docker; última ejecución 100/100 en verde (2026-08-25). El registro detallado está en docs/testing/registro-tests.md.

- **Extracción multimodal real (Agente C):** La extracción multimodal (Agente C) se evaluó con **6 documentos sintéticos** (facturas, acta policial e informe de taller) con *ground truth* conocido. Claude Vision ([claude-sonnet-4-6](#)) acertó **el 100 % de los campos** evaluados (17/17): tipo de documento 6/6, importe 5/5, fecha 6/6. Aun siendo una muestra pequeña sobre documentos sintéticos, confirma la fiabilidad de la extracción en condiciones controladas; una validación productiva requeriría un corpus mayor de documentos reales etiquetados.
- **Demostración CLI/Streamlit:** ejecución de los cinco caminos del flujo con el Chain of Thought visible.

4.9 Conclusiones de la evaluación

- El sistema alcanza una **precisión del 100 % sobre el dataset sintético de 32 casos**, con la matriz de confusión perfectamente diagonal. Este resultado valida la **corrección de la lógica del flujo** (validación → extracción → fraude → cobertura → resolución) y de los criterios de decisión.
- Las **capacidades de IA reales** quedan verificadas: el cribado OFAC bloquea a clientes sancionados, el RAG decide la cobertura recuperando la cláusula de póliza, y la extracción multimodal lee documentos reales.
- La **separación entre lógica determinista y razonamiento por LLM** se confirma acertada: la decisión es estable y reproducible (sin LLM), mientras que Claude enriquece la traza de auditoría y realiza la extracción multimodal cuando está disponible.

4.10 Limitaciones y trabajo futuro

- **Naturaleza del dataset.** Una precisión del 100 % sobre datos sintéticos diseñados para ejercitar la lógica determinista valida el *flujo*, no el rendimiento en producción. Una validación productiva requiere expedientes reales (anonimizados) con la decisión de gestores expertos como *ground truth*.
- **Tamaño.** 32 casos bastan para cubrir los caminos del flujo; se propone ampliar a varios cientos por muestreo de la distribución real.
- **Datos sintéticos.** La lista OFAC, las pólizas (RAG) y los baselines de importe son placeholders; su sustitución por las fuentes reales de Seguros Pepín es trabajo de la fase de producción.
- **Evaluación del VLM.** La extracción multimodal se valida cualitativamente; una métrica cuantitativa (F1 sobre campos extraídos) requiere un corpus etiquetado de documentos.

Bibliografía

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv:2005.11401.
- Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.^a ed.). Pearson.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models*. arXiv:2210.03629.